



内存的分配



连续分配

离散分配



内存的分配

连续分配

离散分配

要把用户程序装入内存，必须给他分配一定内存空间



连续分配方式就是给用户程序分配一个连续的内存空间

程序中代码或数据的逻辑地址相邻，体现在内存空间分配时物理地址的相邻



用户程序1

用户程序2

操作系统

内存



要把用户程序装入内存，必须给他分配一定内存空间

连续分配方式就是给用户程序分配一个连续的内存空间

程序中代码或数据的逻辑地址相邻，体现在内存空间分配时物理地址的相邻





要把用户程序装入内存，必须给他分配一定内存空间

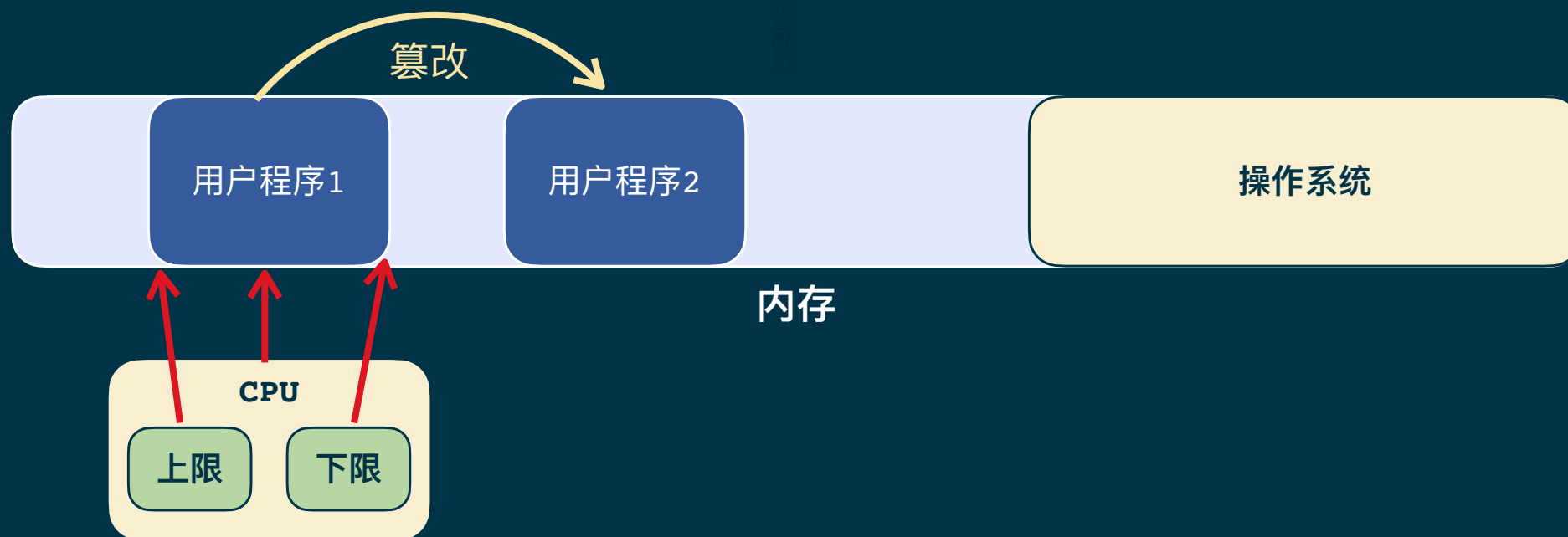
连续分配方式就是给用户程序分配一个连续的内存空间

程序中代码或数据的逻辑地址相邻，体现在内存空间分配时物理地址的相邻

1. CPU中设置一对上下限寄存器，分别记录用户进程在主存中的上下限地址

CPU每次访存的时候都和寄存器对比检查一下有没有越界

程序之间不能互相影响
引入两种方式实现内存保护





要把用户程序装入内存，必须给他分配一定内存空间

连续分配方式就是给用户程序分配一个连续的内存空间

程序中代码或数据的逻辑地址相邻，体现在内存空间分配时物理地址的相邻

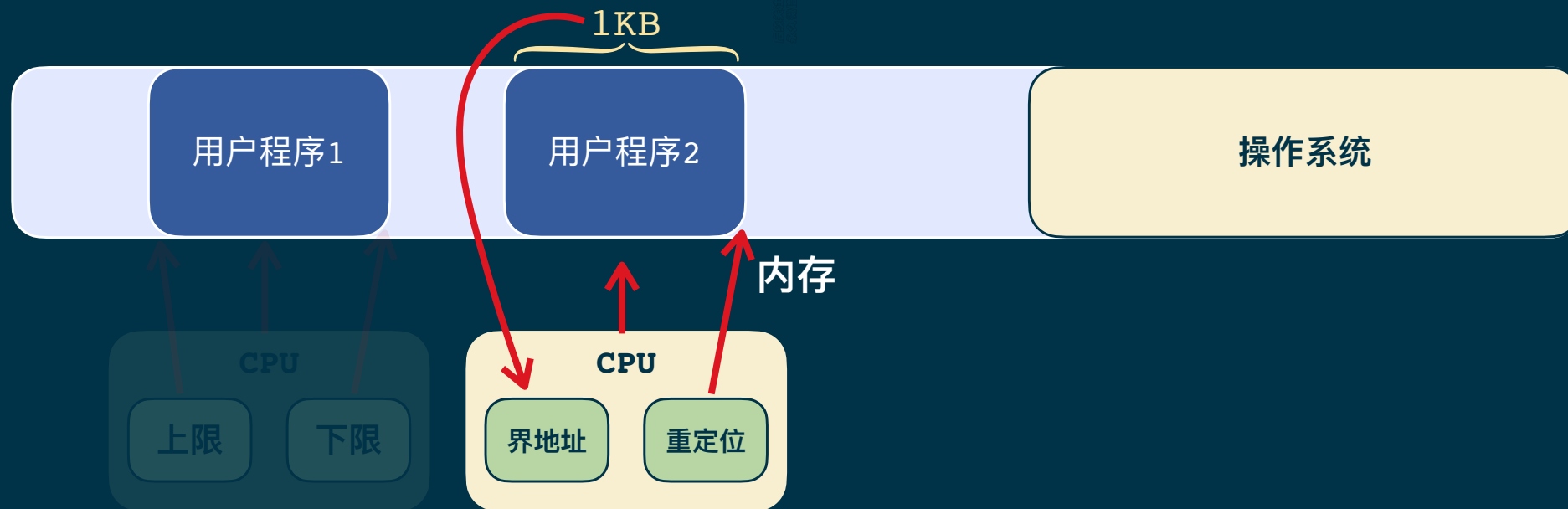
1. CPU中设置一对上下限寄存器，分别记录用户进程在主存中的上下限地址

CPU每次访存的时候都和寄存器对比检查一下有没有越界

2. 采用重定位寄存器和界地址寄存器（界限寄存器）进行越界检查

重定位寄存器中存放的是最小的合法物理内存地址（基地址）

界地址寄存器中存放的是合法范围的大小（界限地址）





要把用户程序装入内存，必须给他分配一定内存空间

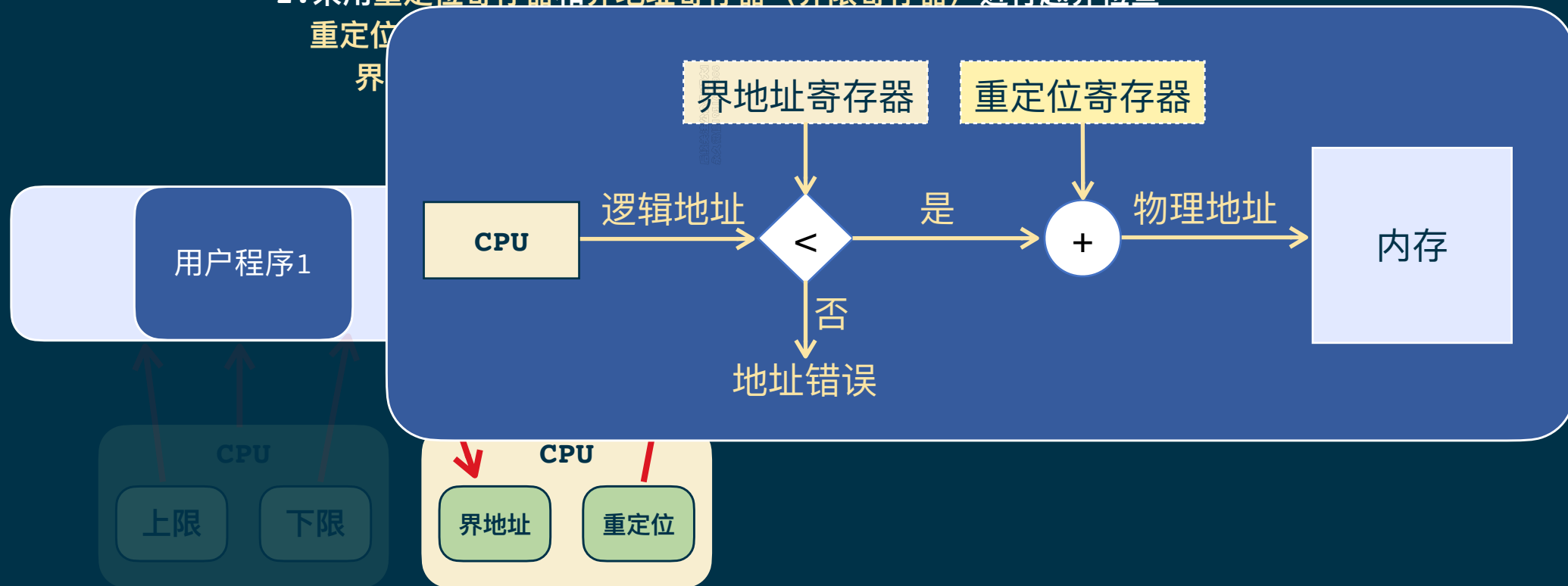
连续分配方式就是给用户程序分配一个连续的内存空间

程序中代码或数据的逻辑地址相邻，体现在内存空间分配时物理地址的相邻

1. CPU中设置一对上下限寄存器，分别记录用户进程在主存中的上下限地址

CPU每次访问的时候都和寄存器对比检查一下有没有越界

2. 采用重定位寄存器和界地址寄存器（界限寄存器）进行越界检查



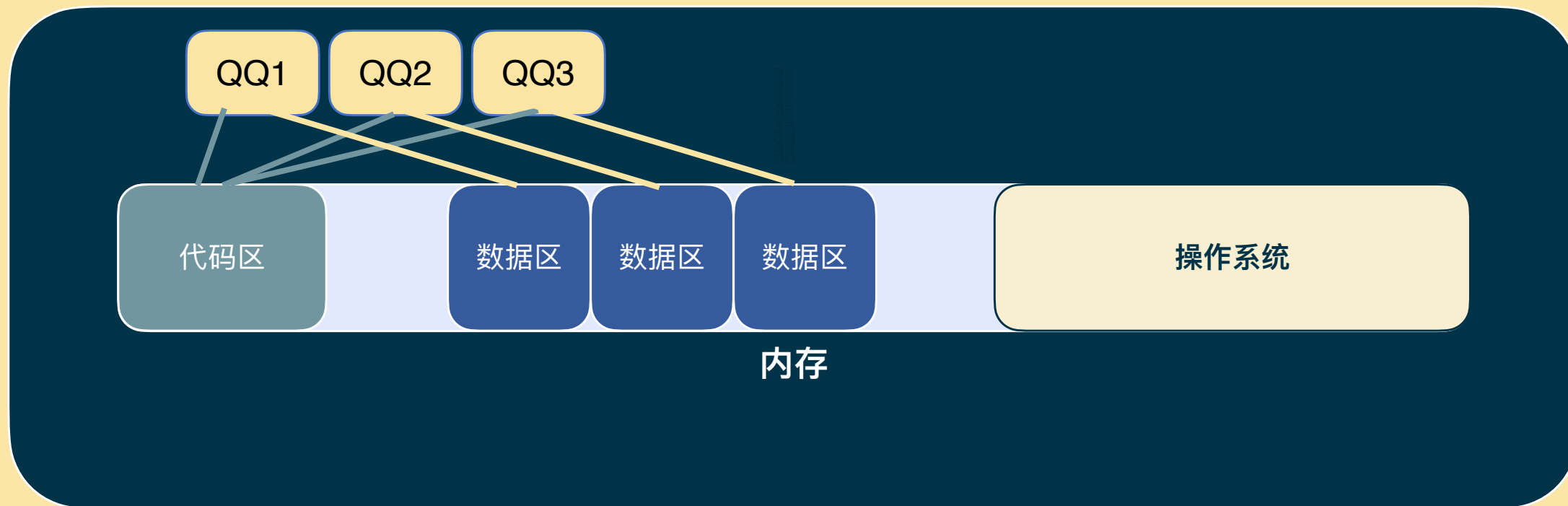


内存共享

不是所有的进程内存空间都适合共享，只有那些只读的区域才能共享

允许多个进程同时访问但不允许被任何进程修改的代码称为**可重入代码（纯代码）**

这种代码适合共享





内存的分配



连续分配

离散分配

单一连续分配

固定分区分配

动态分区分配



内存的分配

连续分配

离散分配

单一连续分配

固定分区分配

动态分区分配

单一连续分配指在**单道程序环境**下，存储器管理方式是吧内存分为系统区和用户区
系统区供OS使用，一般放在内存的低地址部分
而在用户区内存中，**仅装有一道用户程序**，整个内存的用户空间由该程序独占

用户区

系统区

内存



内存的分配

连续分配

离散分配

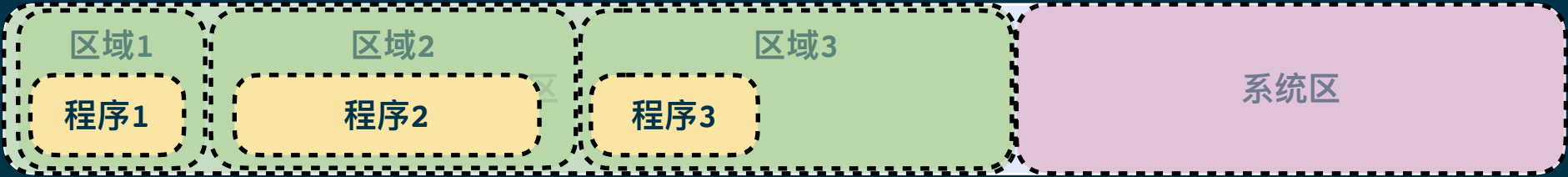
单一连续分配

固定分区分配

动态分区分配

在**多道程序系统**中，为了能在内存中装入多道程序，并且他们之间不会发生互相干扰，把整个用户空间分成几个**固定大小**区域，**每个分区**只装**一个**作业

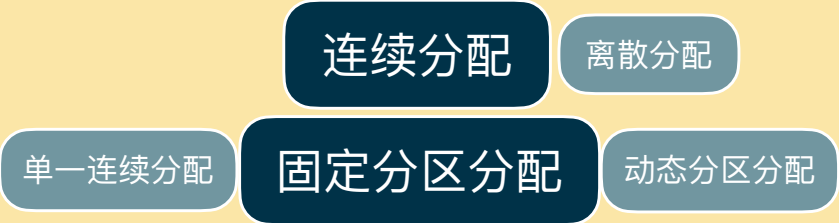
区域可以等大也可以不一样大



内存



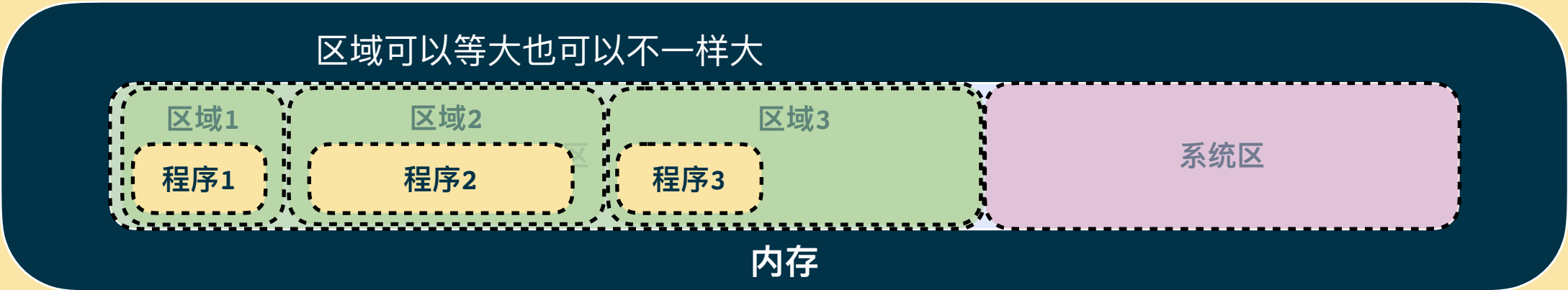
内存的分配



在**多道程序系统**中，为了能在内存中装入多道程序，并且他们之间不会发生互相干扰
把整个用户空间分成几个**固定大小**区域，**每个分区**只装**一个**作业

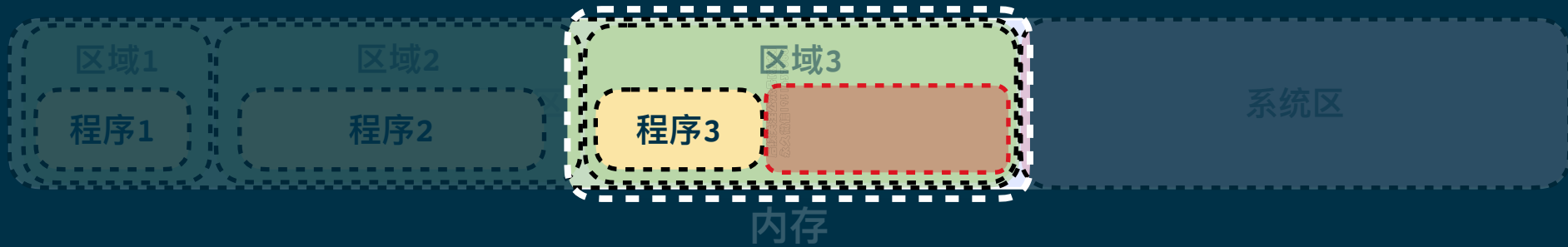
为了便于分配和回收，建立一张分区使用表，按分区大小排队，各表项包括对应分区的起始地址、大小、状态

分区号	大小/KB	起始地址/KB	状态
1	1	20	已分配
2	4	21	已分配
3	32	25	已分配





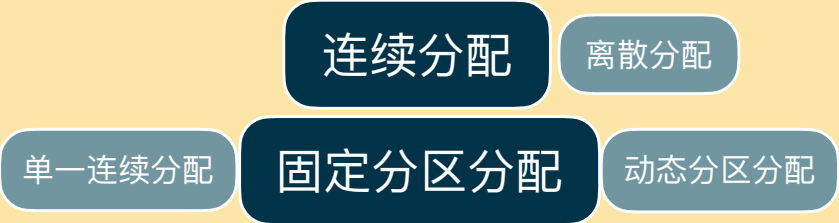
区域可以等大也可以不一样大



这种把空间分配给了程序，
但却未被使用的部分被称为内部碎片



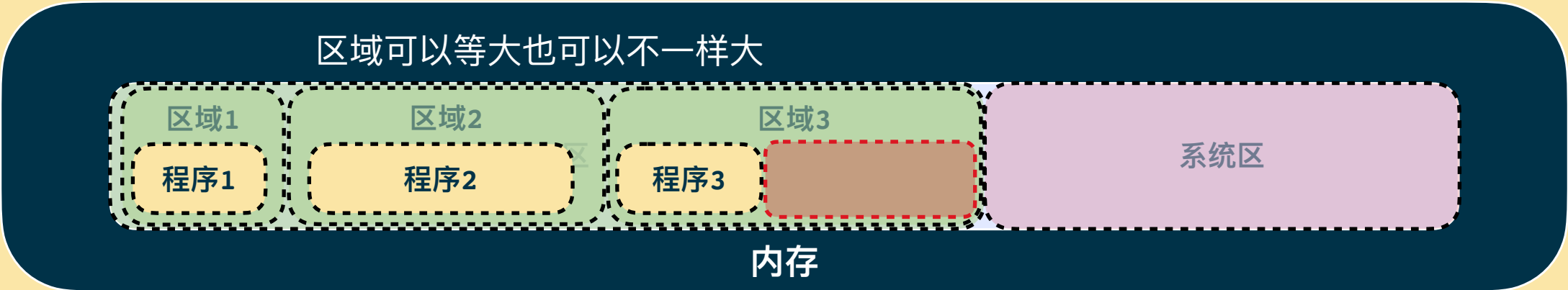
内存的分配



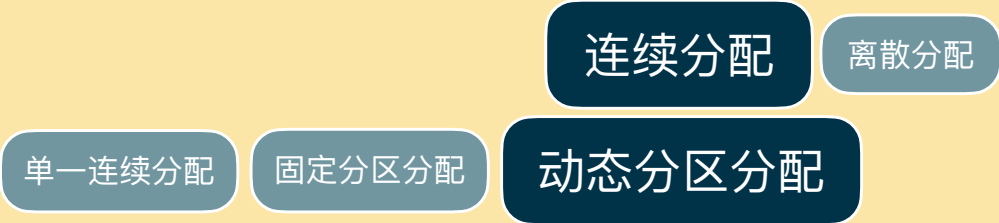
在**多道程序系统**中，为了能在内存中装入多道程序，并且他们之间不会发生互相干扰
把整个用户空间分成几个**固定大小**区域，**每个分区**只装**一个**作业

为了便于分配和回收，建立一张分区使用表，按分区大小排队，各表项包括对应分区的起始地址、大小、状态

分区号	大小/KB	起始地址/KB	状态
1	1	20	已分配
2	4	21	已分配
3	32	25	已分配



内存的分配



空闲分区表

系统中设置一张空闲分区表，记录每个空闲分区的情况
每个空闲分区占用一个表目，表目中包括分区号、分区大小和分区起始地址等数据

分区号	分区大小 (KB)	分区始址	状态
1	50	85	空闲
2	32	155	空闲
3	70	275	空闲
4	60	532	空闲
5	...	...	...



内存的分配

连续分配

离散分配

单一连续分配

固定分区分配

动态分区分配

分配算法



首次适应算法

找第一个能满足大小的分区分配

邻近适应算法

对首次适应的改进
从查找结束位置开始继续查找

最佳适应算法

找**最小的**能满足大小的空闲分区分配

最坏适应算法

找**最大的**能满足大小的空闲分区分配

内存的分配

连续分配

离散分配

单一连续分配

固定分区分配

动态分区分配

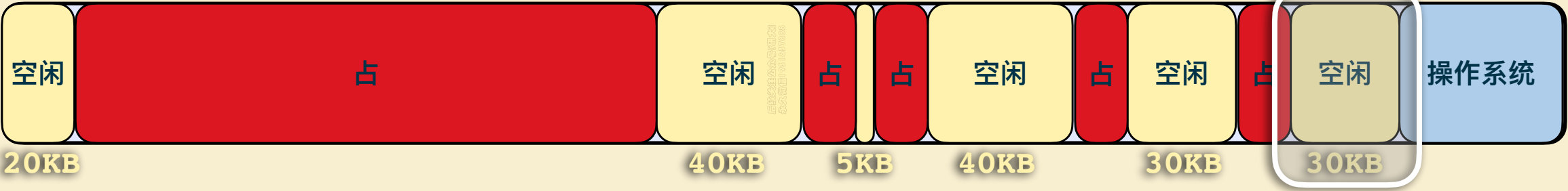
分配算法

首次适应算法

找第一个能满足大小的分区分配

已被占用

空闲



35KB

新作业到达

分区号	分区大小/KB	分区始址/KB	状态
1	30	1070	空闲
2	30	1106	空闲
3	40	1140	空闲
4	5	1185	空闲
5	40	1192	空闲
6	20	1600	空闲

内存的分配

连续分配

离散分配

单一连续分配

固定分区分配

动态分区分配

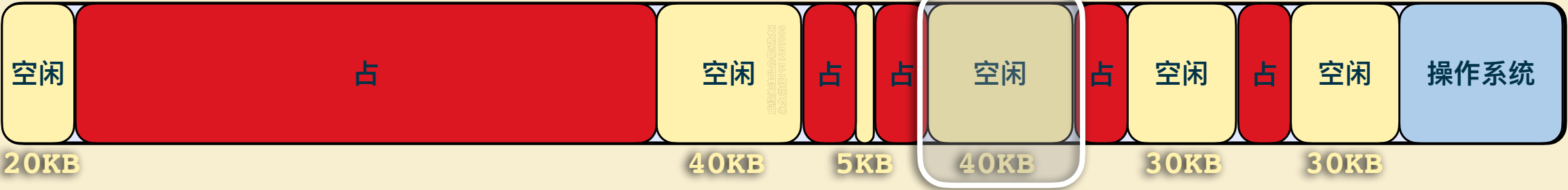
分配算法

首次适应算法

找第一个能满足大小的分区分配

已被占用

空闲



35KB

新作业到达

分区号	分区大小/KB	分区始址/KB	状态
1	30	1070	空闲
2	30	1106	空闲
3	40	1140	空闲
4	5	1185	空闲
5	40	1192	空闲
6	20	1600	空闲

内存的分配

连续分配

离散分配

单一连续分配

固定分区分配

动态分区分配

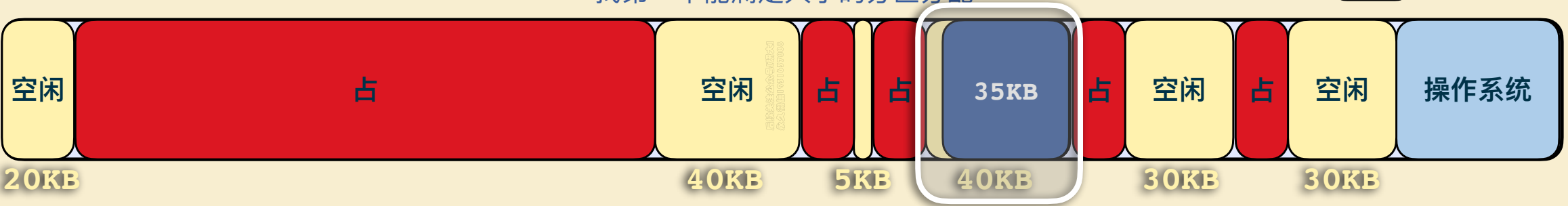
分配算法

首次适应算法

找第一个能满足大小的分区分配

已被占用

空闲



分区号	分区大小/KB	分区始址/KB	状态
1	30	1070	空闲
2	30	1106	空闲
3	40	1140	空闲
4	5	1185	空闲
5	40	1192	空闲
6	20	1600	空闲

内存的分配

连续分配

离散分配

单一连续分配

固定分区分配

动态分区分配

分配算法

首次适应算法

找第一个能满足大小的分区分配

已被占用

空闲



分区号	分区大小/KB	分区始址/KB	状态
1	30	1070	空闲
2	30	1106	空闲
3	5	1175	空闲
4	5	1185	空闲
5	40	1192	空闲
6	20	1600	空闲



内存的分配

连续分配

离散分配

单一连续分配

固定分区分配

动态分区分配

分配算法



首次适应算法

找第一个能满足大小的分区分配

邻近适应算法

对首次适应的改进
从查找结束位置开始继续查找

最佳适应算法

找**最小的**能满足大小的空闲分区分配

最坏适应算法

找**最大的**能满足大小的空闲分区分配

内存的分配

连续分配

离散分配

单一连续分配

固定分区分配

动态分区分配

分配算法

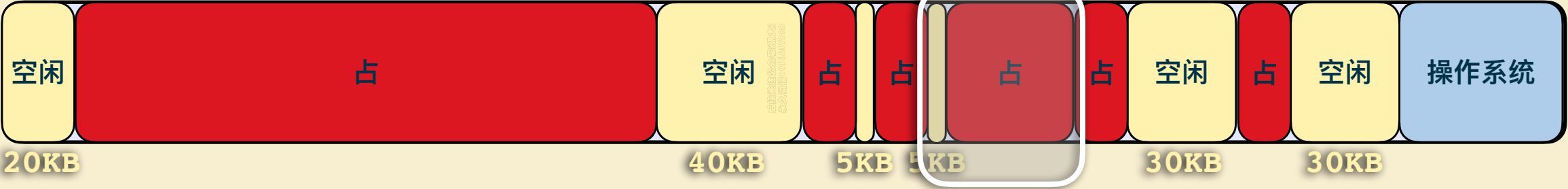
邻近适应算法

对首次适应的改进

从查找结束位置开始继续查找

已被占用

空闲



15 KB

新作业到达

分区号	分区大小/KB	分区始址/KB	状态
1	30	1070	空闲
2	30	1106	空闲
3	5	1175	空闲
4	5	1185	空闲
5	40	1192	空闲
6	20	1600	空闲

上次分配位置

内存的分配

连续分配

离散分配

单一连续分配

固定分区分配

动态分区分配

分配算法

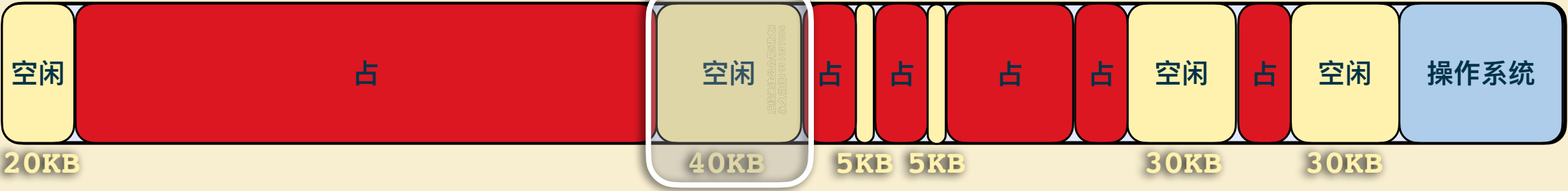
邻近适应算法

对首次适应的改进

从查找结束位置开始继续查找

已被占用

空闲



15 KB

新作业到达

分区号	分区大小/KB	分区始址/KB	状态
1	30	1070	空闲
2	30	1106	空闲
3	5	1175	空闲
4	5	1185	空闲
5	40	1192	空闲
6	20	1600	空闲

上次分配位置

内存的分配

连续分配

离散分配

单一连续分配

固定分区分配

动态分区分配

分配算法

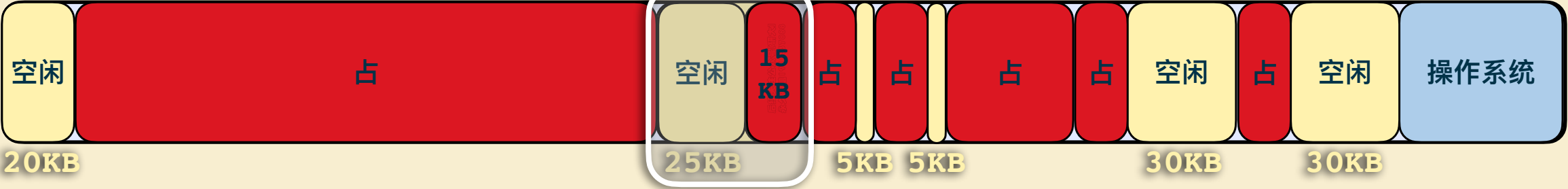
邻近适应算法

对首次适应的改进

从查找结束位置开始继续查找

已被占用

空闲



分区号	分区大小/KB	分区始址/KB	状态
1	30	1070	空闲
2	30	1106	空闲
3	5	1175	空闲
4	5	1185	空闲
5	15	1207	空闲
6	20	1600	空闲

上次分配位置



内存的分配

连续分配

离散分配

单一连续分配

固定分区分配

动态分区分配

分配算法



首次适应算法

找第一个能满足大小的分区分配

邻近适应算法

对首次适应的改进
从查找结束位置开始继续查找

最佳适应算法

找**最小的**能满足大小的空闲分区分配

最坏适应算法

找**最大的**能满足大小的空闲分区分配

内存的分配



连续分配 离散分配

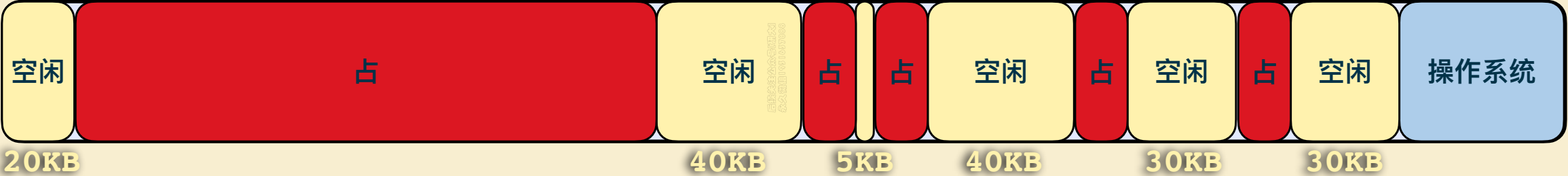
单一连续分配 固定分区分配 动态分区分配

分配算法

最佳适应算法

找最小的能满足大小的空闲分区分配

已被占用
空闲



15 KB

新作业到达

分区号	分区大小/KB	分区始址/KB	状态
1	5	1185	空闲
2	20	1600	空闲
3	30	1070	空闲
4	30	1106	空闲
5	40	1140	空闲
6	40	1192	空闲

空闲分区按容量递增排列

内存的分配



连续分配 离散分配

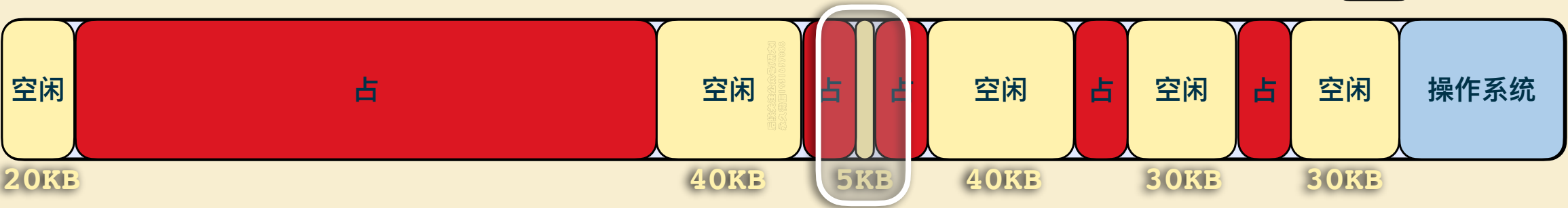
单一连续分配 固定分区分配 动态分区分配

分配算法

最佳适应算法

找最小的能满足大小的空闲分区分配

已被占用
空闲



15 KB
新作业到达

分区号	分区大小/KB	分区始址/KB	状态
1	5	1185	空闲
2	20	1600	空闲
3	30	1070	空闲
4	30	1106	空闲
5	40	1140	空闲
6	40	1192	空闲

内存的分配



连续分配

离散分配

单一连续分配

固定分区分配

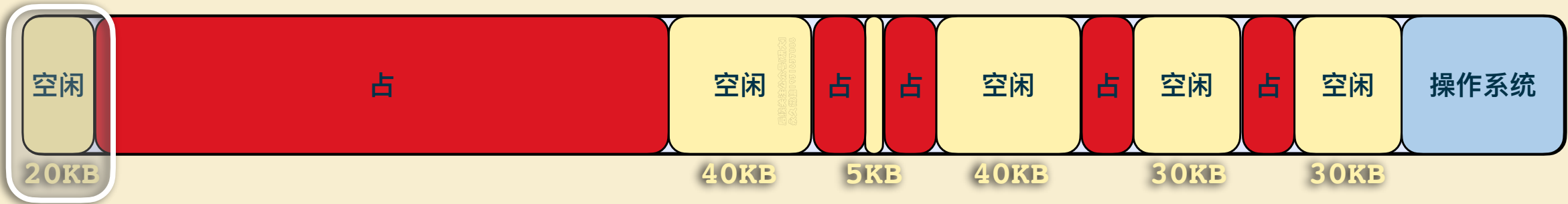
动态分区分配

分配算法

最佳适应算法

找最小的能满足大小的空闲分区分配

已被占用
空闲



15
KB

新作业到达

分区号	分区大小/KB	分区始址/KB	状态
1	5	1185	空闲
2	20	1600	空闲
3	30	1070	空闲
4	30	1106	空闲
5	40	1140	空闲
6	40	1192	空闲

内存的分配



连续分配 离散分配

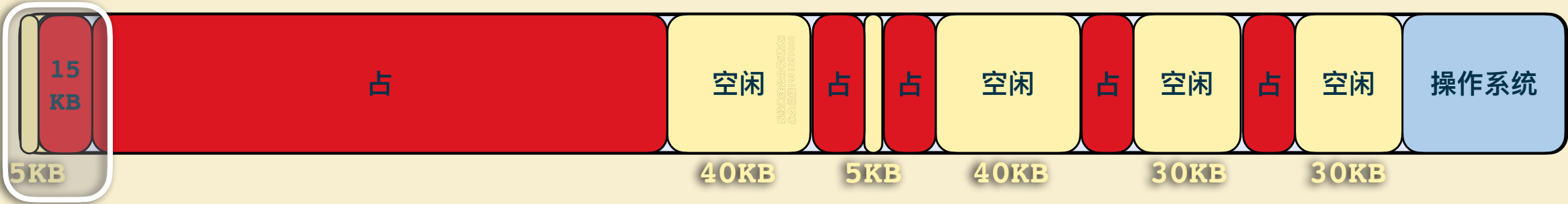
单一连续分配 固定分区分配 动态分区分配

分配算法

最佳适应算法

找最小的能满足大小的空闲分区分配

已被占用
空闲



分区号	分区大小/KB	分区始址/KB	状态
1	5	1185	空闲
2	5	1615	空闲
3	30	1070	空闲
4	30	1106	空闲
5	40	1140	空闲
6	40	1192	空闲

如果有新的空闲分区的话
需要重新按照大小排序



内存的分配

连续分配

离散分配

单一连续分配

固定分区分配

动态分区分配

分配算法



首次适应算法

找第一个能满足大小的分区分配

邻近适应算法

对首次适应的改进
从查找结束位置开始继续查找

最佳适应算法

找**最小的**能满足大小的空闲分区分配

最坏适应算法

找**最大的**能满足大小的空闲分区分配

内存的分配



连续分配

离散分配

单一连续分配

固定分区分配

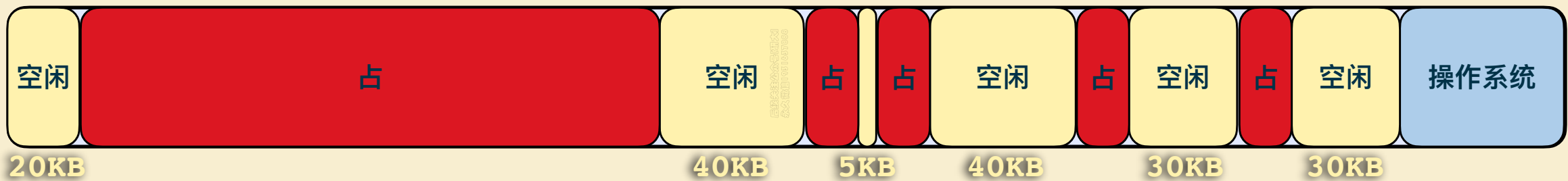
动态分区分配

分配算法

最坏适应算法

找最大的能满足大小的空闲分区分配

已被占用
空闲



5KB新作业到达

分区号	分区大小/KB	分区始址/KB	状态
1	40	1140	空闲
2	40	1192	空闲
3	30	1070	空闲
4	30	1106	空闲
5	20	1600	空闲
6	5	1185	空闲

空闲分区按容量递减排列

内存的分配

连续分配

离散分配

单一连续分配

固定分区分配

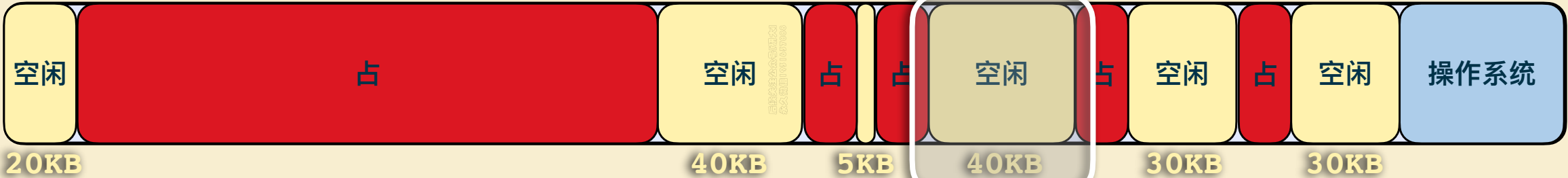
动态分区分配

分配算法

最坏适应算法

找最大的能满足大小的空闲分区分配

 已被占用
 空闲



5KB新作业到达

分区号	分区大小/KB	分区始址/KB	状态
1	40	1140	空闲
2	40	1192	空闲
3	30	1070	空闲
4	30	1106	空闲
5	20	1600	空闲
6	5	1185	空闲

内存的分配

连续分配

离散分配

单一连续分配

固定分区分配

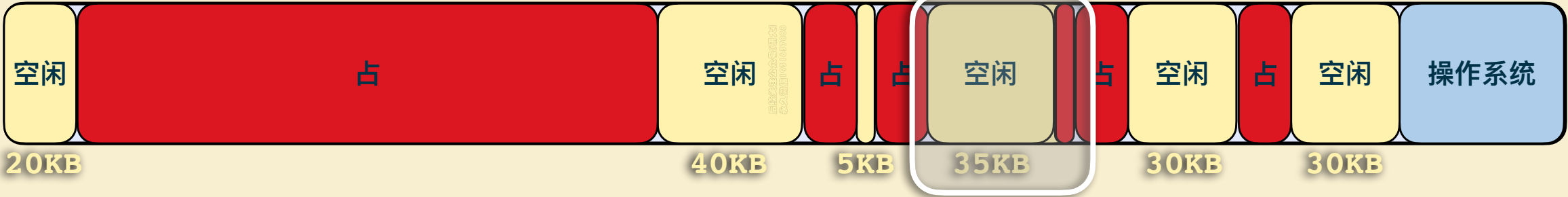
动态分区分配

分配算法

最坏适应算法

找最大的能满足大小的空闲分区分配

 已被占用
 空闲



分区号	分区大小/KB	分区始址/KB	状态
1	35	1145	空闲
2	40	1192	空闲
3	30	1070	空闲
4	30	1106	空闲
5	20	1600	空闲
6	5	1185	空闲

新分区需要排序
调整空闲分区表

内存的分配

连续分配

离散分配

单一连续分配

固定分区分配

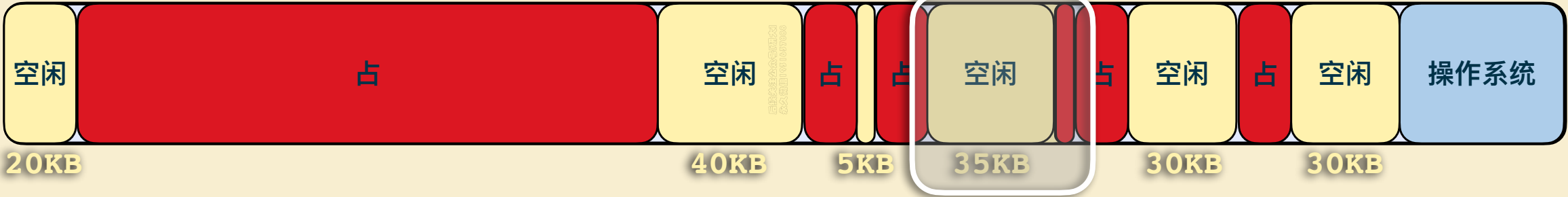
动态分区分配

分配算法

最坏适应算法

找最大的能满足大小的空闲分区分配

 已被占用
 空闲



分区号	分区大小/KB	分区始址/KB	状态
1	40	1192	空闲
2	35	1145	空闲
3	30	1070	空闲
4	30	1106	空闲
5	20	1600	空闲
6	5	1185	空闲

新分区需要排序
调整空闲分区表



内存的分配

连续分配

离散分配

单一连续分配

固定分区分配

动态分区分配

分配算法



首次适应算法

找第一个能满足大小的分区分配

邻近适应算法

对首次适应的改进
从查找结束位置开始继续查找

最佳适应算法

找**最小的**能满足大小的空闲分区分配

最坏适应算法

找**最大的**能满足大小的空闲分区分配

内存的分配



连续分配

离散分配

单一连续分配

固定分区分配

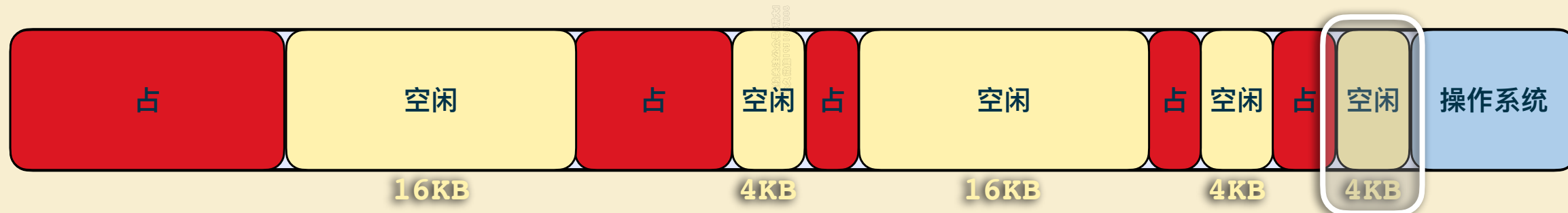
动态分区分配

伙伴系统

伙伴系统规定：所有分区均为2的k次幂，k是正整数

已被占用
空闲

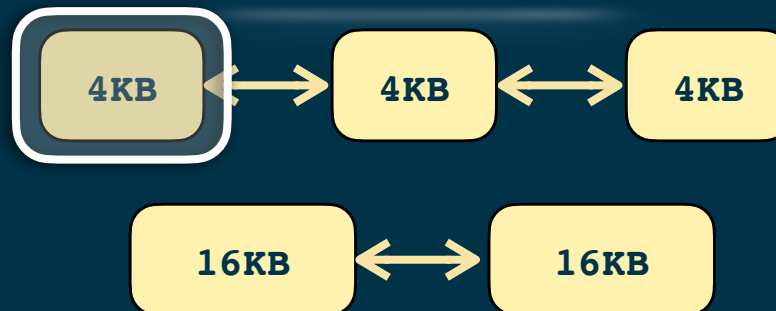
所有相同大小的空闲分区，为他们单独设立一个空闲分区双向链表需要给进程分配大小为n的空间时
先计算 $2^{i-1} < n \leq 2^i$ ，在分区大小为 2^i 的空闲分区链表中找，找到就分配



3
KB

3KB新作业到达

空闲分区双向链表



内存的分配



连续分配

离散分配

单一连续分配

固定分区分配

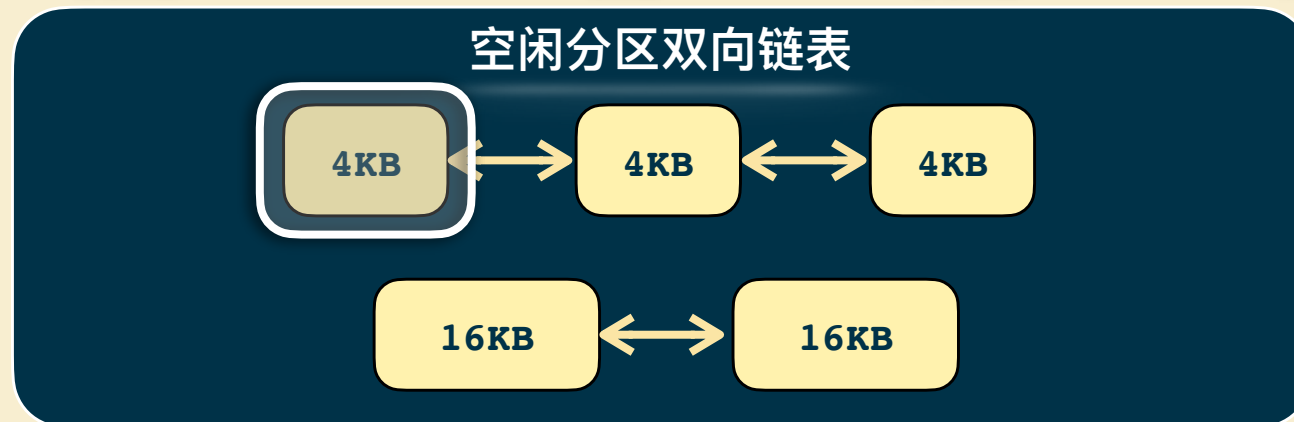
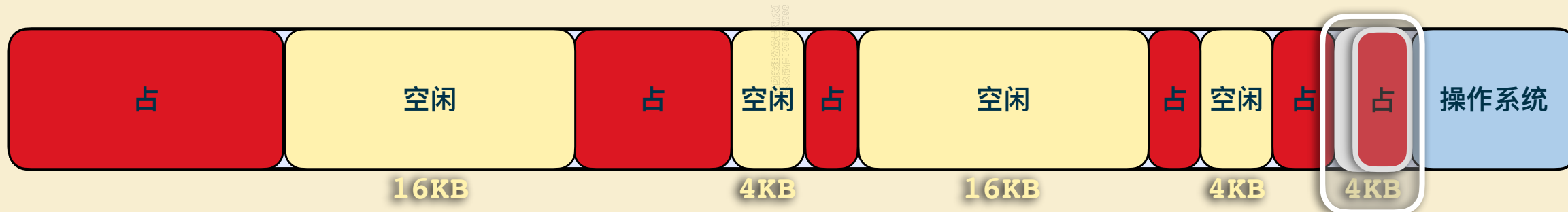
动态分区分配

伙伴系统

伙伴系统规定：所有分区均为2的k次幂，k是正整数

所有相同大小的空闲分区，为他们单独设立一个空闲分区双向链表需要给进程分配大小为n的空间时
先计算 $2^{i-1} < n \leq 2^i$ ，在分区大小为 2^i 的空闲分区链表中找，找到就分配

已被占用
空闲



4KB分区被分配
1KB为内部碎片

内存的分配

BOK操作系统



连续分配

离散分配

单一连续分配

固定分区分配

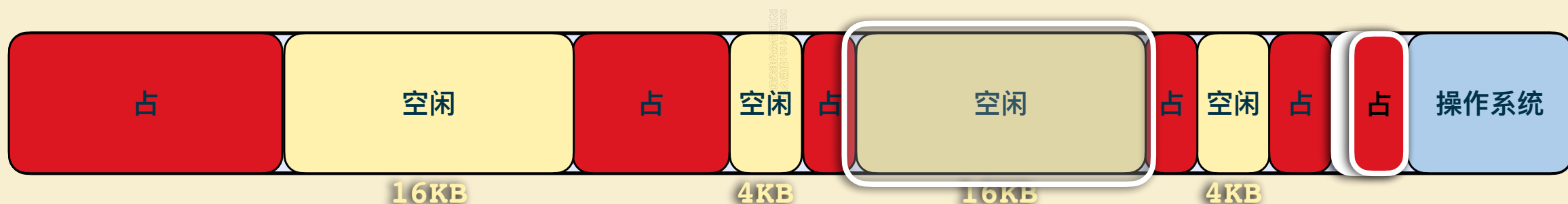
动态分区分配

伙伴系统

已被占用
空闲

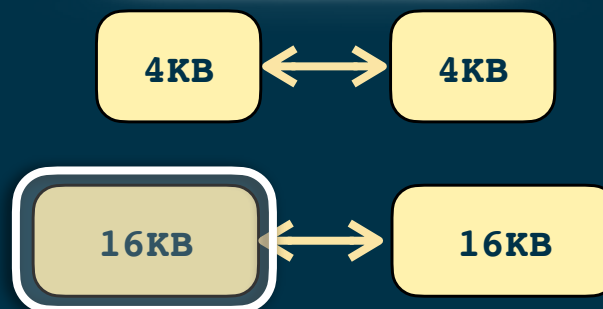
伙伴系统规定：所有分区均为2的k次幂，k是正整数

所有相同大小的空闲分区，为他们单独设立一个空闲分区双向链表需要给进程分配大小为n的空间时
先计算 $2^{i-1} < n \leq 2^i$ ，在分区大小为 2^i 的空闲分区链表中找，找到就分配



5 KB
5KB新作业到达

空闲分区双向链表



当找不到 $2^{i-1} < n \leq 2^i$ ，即8KB空间时
就在大小 2^{i+1} (16KB)的空闲分区链表中找
如果找到，把该分区分成两个相同大小分区
(称为一对伙伴)
其中一个分区用于分配
再把另一个分区加入大小为 2^i 的空闲分区链表中

内存的分配



连续分配

离散分配

单一连续分配

固定分区分配

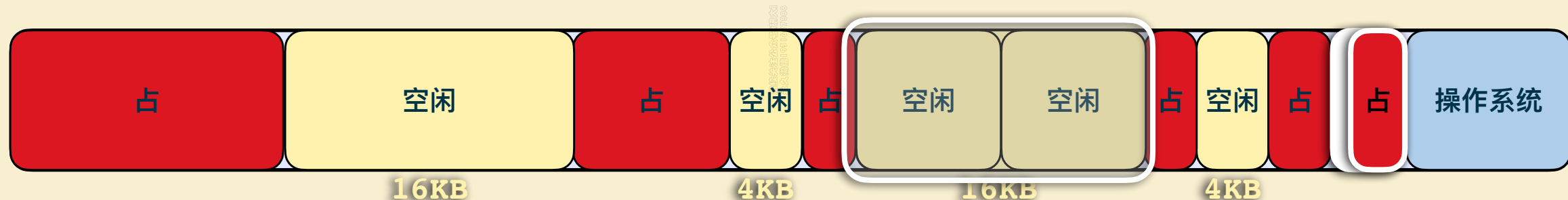
动态分区分配

伙伴系统

已被占用
空闲

伙伴系统规定：所有分区均为2的k次幂，k是正整数

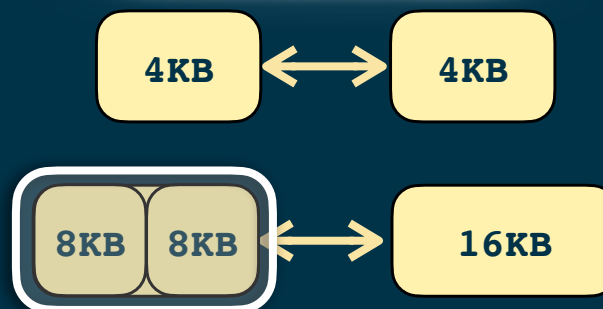
所有相同大小的空闲分区，为他们单独设立一个空闲分区双向链表需要给进程分配大小为n的空间时
先计算 $2^{i-1} < n \leq 2^i$ ，在分区大小为 2^i 的空闲分区链表中找，找到就分配



5
KB

5KB新作业到达

空闲分区双向链表



当找不到 $2^{i-1} < n \leq 2^i$ ，即8KB空间时
就在大小 2^{i+1} (16KB)的空闲分区链表中找
如果找到，把该分区分成两个相同大小分区
(称为一对伙伴)
其中一个分区用于分配
再把另一个分区加入大小为 2^i 的空闲分区链表中

内存的分配

BOK操作系统



连续分配

离散分配

单一连续分配

固定分区分配

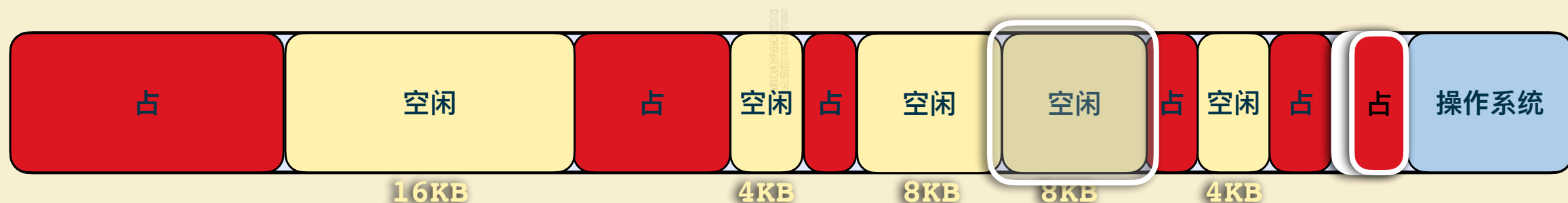
动态分区分配

伙伴系统

伙伴系统规定：所有分区均为2的k次幂，k是正整数

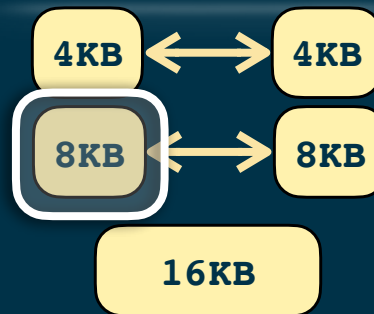
已被占用
空闲

所有相同大小的空闲分区，为他们单独设立一个空闲分区双向链表需要给进程分配大小为n的空间时
先计算 $2^{i-1} < n \leq 2^i$ ，在分区大小为 2^i 的空闲分区链表中找，找到就分配



5 KB
5KB新作业到达

空闲分区双向链表



当找不到 $2^{i-1} < n \leq 2^i$ ，即8KB空间时
就在大小 2^{i+1} (16KB) 的空闲分区链表中找
如果找到，把该分区分成两个相同大小分区
(称为一对伙伴)
其中一个分区用于分配
再把另一个分区加入大小为 2^i 的空闲分区链表中

内存的分配



连续分配 离散分配

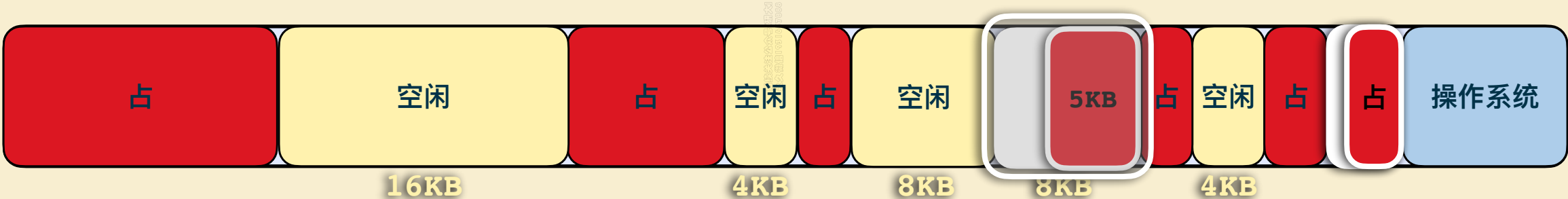
单一连续分配 固定分区分配 动态分区分配

伙伴系统

已被占用
空闲

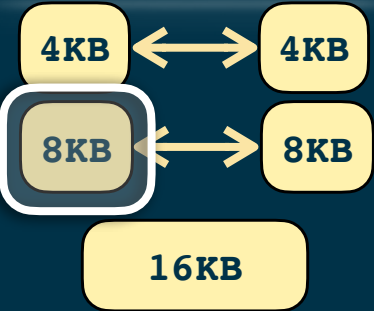
伙伴系统规定：所有分区均为2的k次幂，k是正整数

所有相同大小的空闲分区，为他们单独设立一个空闲分区双向链表需要给进程分配大小为n的空间时
先计算 $2^{i-1} < n \leq 2^i$ ，在分区大小为 2^i 的空闲分区链表里找，找到就分配



5KB新作业到达

空闲分区双向链表



8KB作为整体被分配
3KB为内部碎片

$2^{i-1} < n \leq 2^i$ ，即8KB空间时
就在大小为 2^{i-1} (16KB)的空闲分区链表里找
如果找到，把该分区分成两个相同大小分区
(称为一对伙伴)
其中一个分区用于分配
再把另一个分区加入大小为 2^i 的空闲分区链表中

内存的分配

BOK操作系统



连续分配

离散分配

单一连续分配

固定分区分配

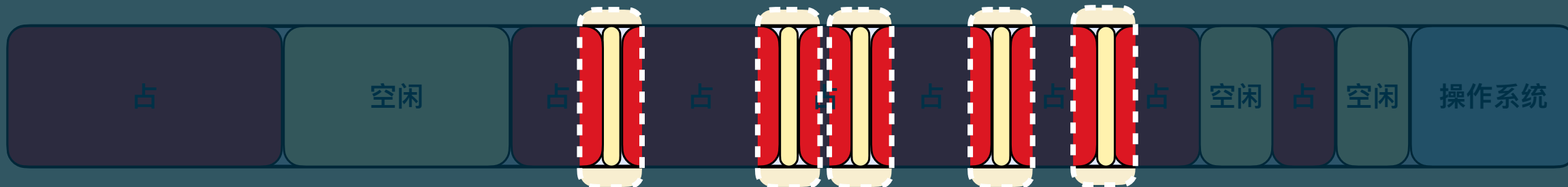
动态分区分配

紧凑技术

已被占用
空闲

整理外部碎片

这种小的空间难以被分配出去
被称为外部碎片
可以通过紧凑技术来解决
但是需要动态重定位寄存器的支持，而且费时



内存的分配



连续分配

离散分配

单一连续分配

固定分区分配

动态分区分配

紧凑技术

 已被占用
 空闲

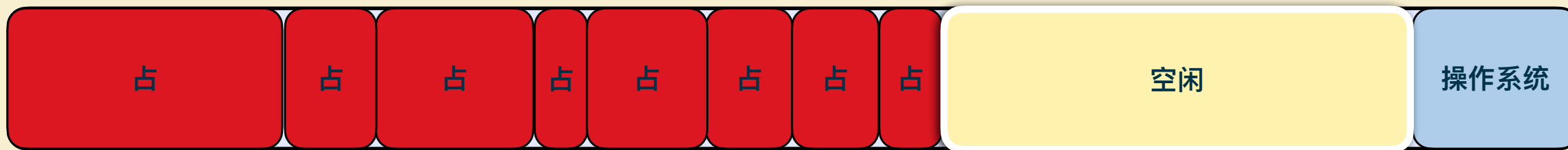
整理外部碎片

这种小的空间难以被分配出去

被称为**外部碎片**

可以通过紧凑技术来解决

但是需要动态重定位寄存器的支持，而且费时



内存的分配

连续分配

离散分配

单一连续分配

固定分区分配

动态分区分配

分区分配操作

已被占用

空闲

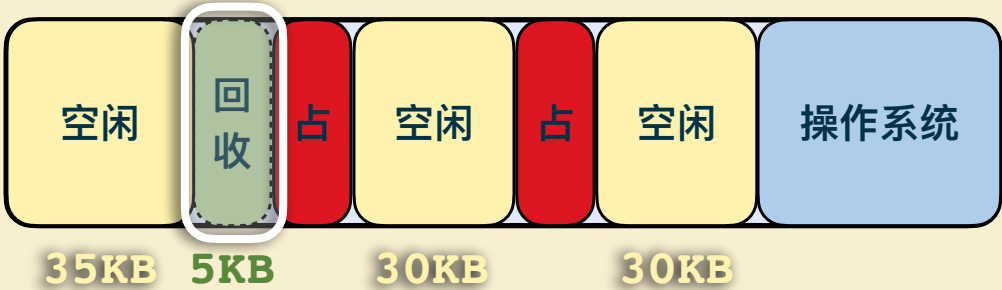
分配内存：根据系统的分配算法，找到空闲分区表中对应的分区

如果进程需要空间<空闲分区空间，则把空闲分区分割后，多余的空闲空间留在空闲分区表中

根据需求排序，其余分配给发出申请的进程

回收内存：进程运行完毕释放内存时

系统根据回收区的首址，从空闲表中找到对应插入点，根据情况选择是否与其他空闲分区合并



分区号	分区大小/KB	分区始址/KB	状态
1	35	1145	空闲
2	30	1070	空闲
3	30	1106	空闲

内存的分配

连续分配

离散分配

单一连续分配

固定分区分配

动态分区分配

分区分配操作

已被占用

空闲

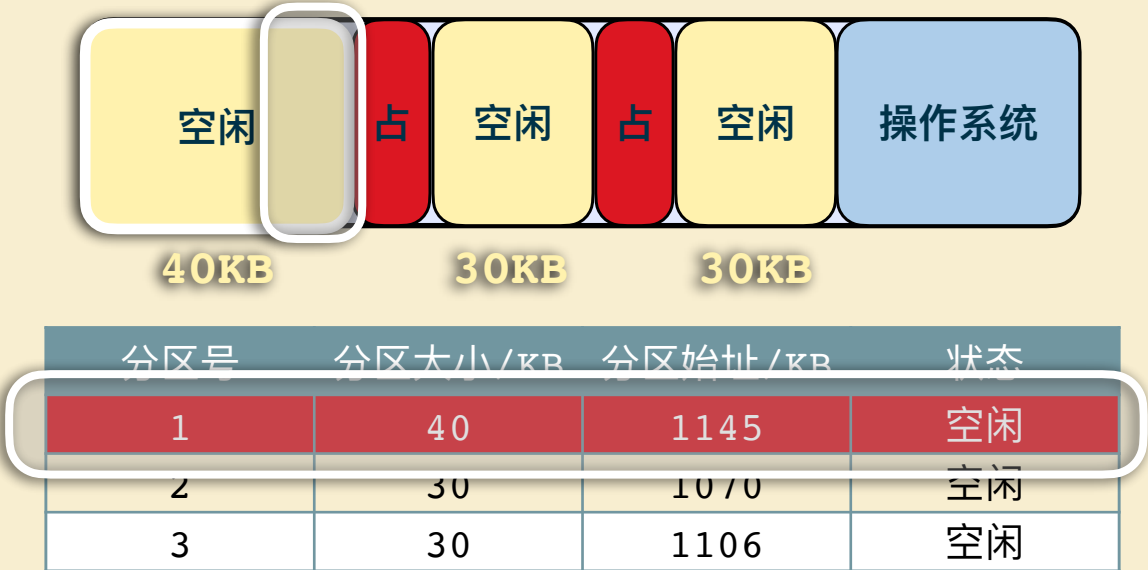
分配内存：根据系统的分配算法，找到空闲分区表中对应的分区

如果进程需要空间<空闲分区空间，则把空闲分区分割后，多余的空闲空间留在空闲分区表中

根据需求排序，其余分配给发出申请的进程

回收内存：进程运行完毕释放内存时

系统根据回收区的首址，从空闲表中找到对应插入点，根据情况选择是否与其他空闲分区合并



内存的分配

连续分配

离散分配

单一连续分配

固定分区分配

动态分区分配

分区分配操作

已被占用

空闲

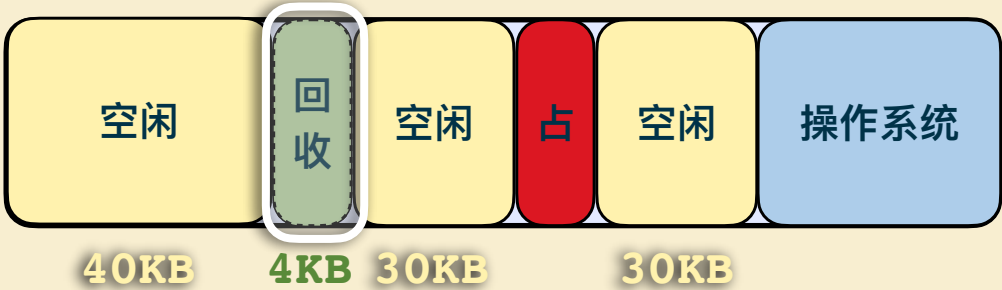
分配内存：根据系统的分配算法，找到空闲分区表中对应的分区

如果进程需要空间 $<$ 空闲分区空间，则把空闲分区分割后，多余的空闲空间留在空闲分区表中

根据需求排序，其余分配给发出申请的进程

回收内存：进程运行完毕释放内存时

系统根据回收区的首址，从空闲表中找到对应插入点，根据情况选择是否与其他空闲分区合并



分区号	分区大小/KB	分区始址/KB	状态
1	40	1145	空闲
2	30	1070	空闲
3	30	1106	空闲

如果回收和前后分区相连
则需要考虑合并表项

内存的分配

连续分配

离散分配

单一连续分配

固定分区分配

动态分区分配

分区分配操作

已被占用

空闲

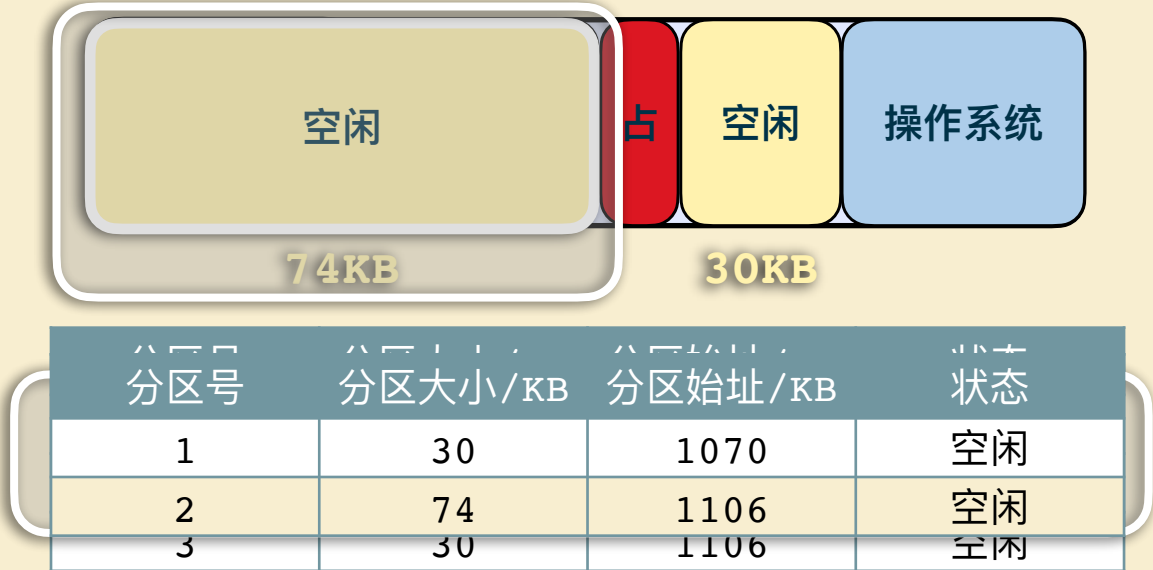
分配内存：根据系统的分配算法，找到空闲分区表中对应的分区

如果进程需要空间<空闲分区空间，则把空闲分区分割后，多余的空闲空间留在空闲分区表中

根据需求排序，其余分配给发出申请的进程

回收内存：进程运行完毕释放内存时

系统根据回收区的首址，从空闲表中找到对应插入点，根据情况选择是否与其他空闲分区合并



如果回收和前后分区相连
则需要考虑合并表项



内存的分配



内存的分配



连续分配

离散分配

分页存储

分段存储

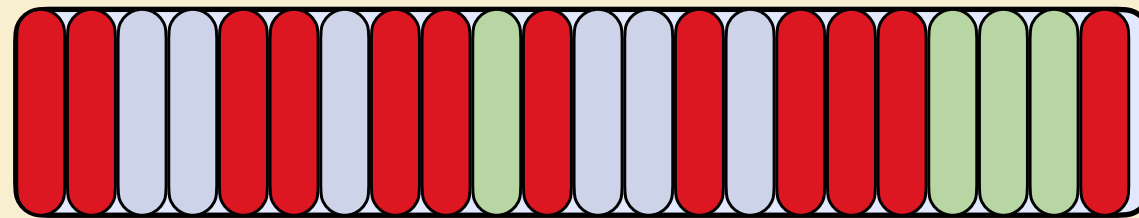
段页式存储

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销，所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了。

连续分配下的内存



分页存储下的内存



用户程序1

用户程序2

用户程序3

内存的分配



连续分配

离散分配

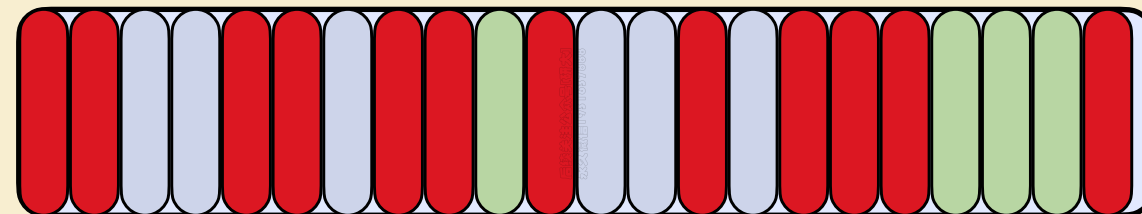
分页存储

分段存储

段页式存储

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销，所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了。

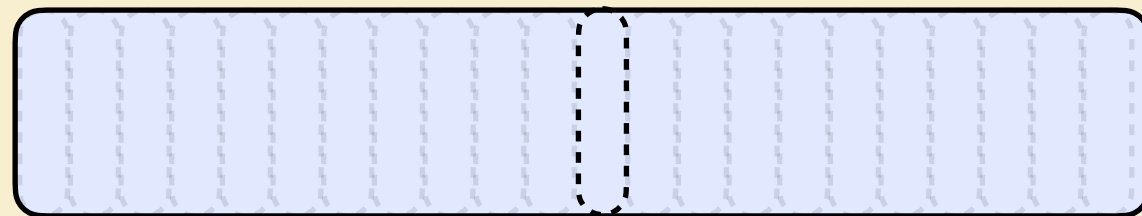
分页存储下的内存



用户程序1

用户程序2

用户程序3



被分割成的固定大小的区域被称为“页”或“页面”

内存的分配



连续分配

离散分配

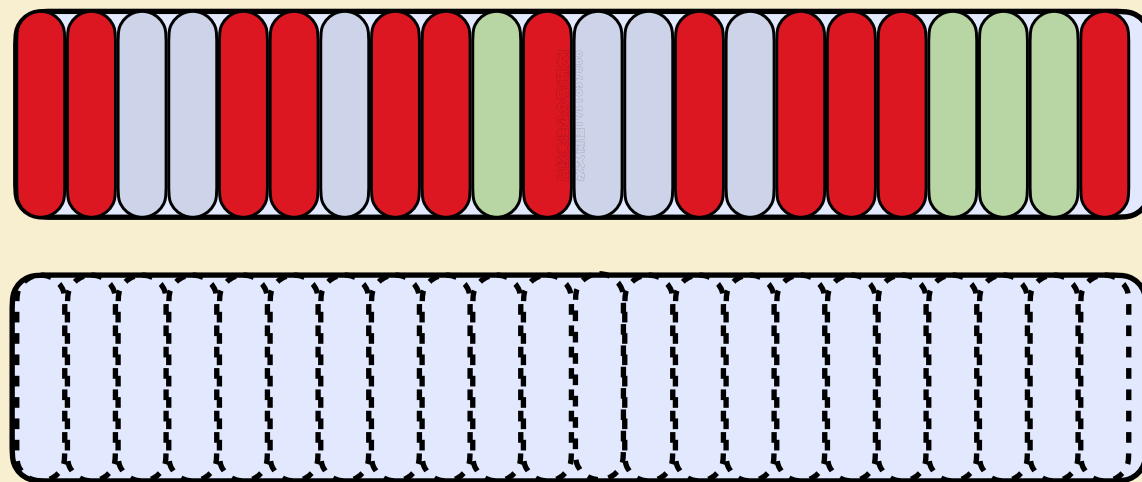
分页存储

分段存储

段页式存储

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销，所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了。

分页存储下的内存



用户程序1
用户程序2
用户程序3

对应的内存也要被分割成“物理块”或“页框”，这样就可以放下“页”
被分割成的固定大小的区域被称为“页”或“页面”

内存的分配



连续分配

离散分配

分页存储

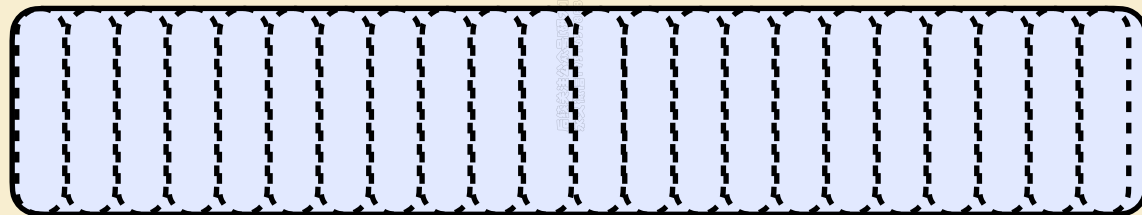
分段存储

段页式存储

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销，所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了。

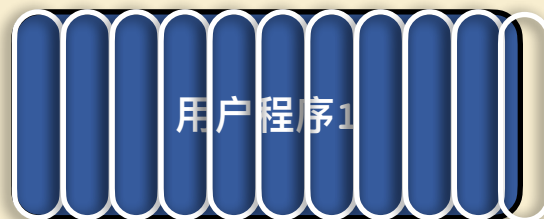
对应的内存也要被分割成“物理块”或“页框”，这样就可以放下“页”
被分割成的固定大小的区域被称为“页”或“页面”

分页存储下的内存



地址变换机构

用户程序1



内存的分配



连续分配

离散分配

分页存储

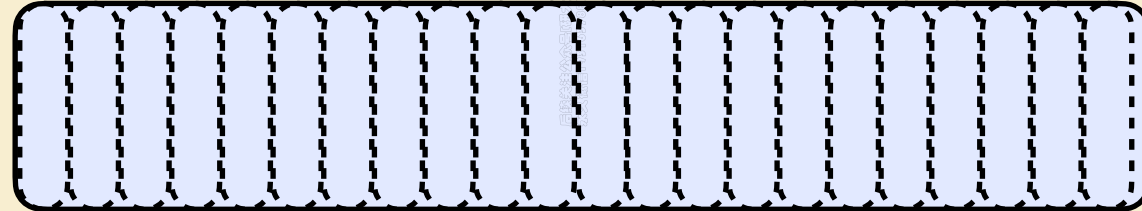
分段存储

段页式存储

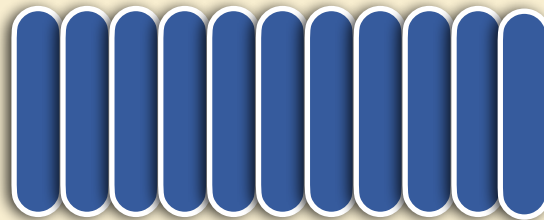
连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销，所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了。

对应的内存也要被分割成“物理块”或“页框”，这样就可以放下“页”
被分割成的固定大小的区域被称为“页”或“页面”

分页存储下的内存



地址变换机构



用户程序1

内存的分配



连续分配

离散分配

分页存储

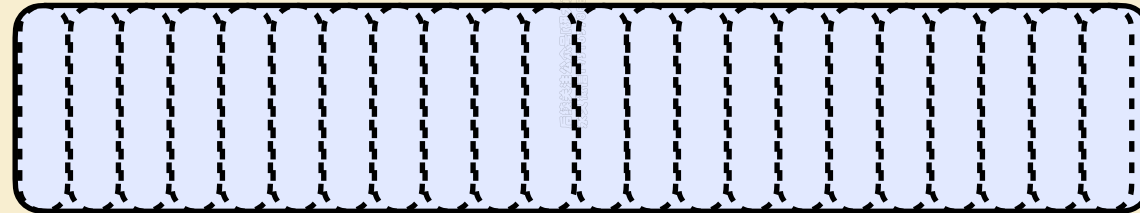
分段存储

段页式存储

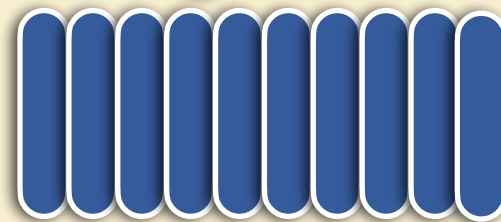
连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销，所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了。

对应的内存也要被分割成“物理块”或“页框”，这样就可以放下“页”
被分割成的固定大小的区域被称为“页”或“页面”

分页存储下的内存



地址变换机构



用户程序1

内存的分配



连续分配

离散分配

分页存储

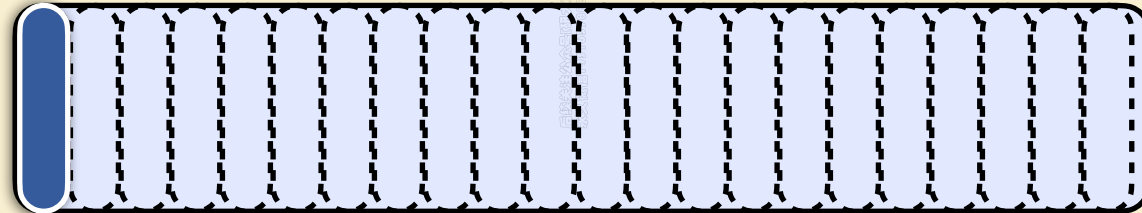
分段存储

段页式存储

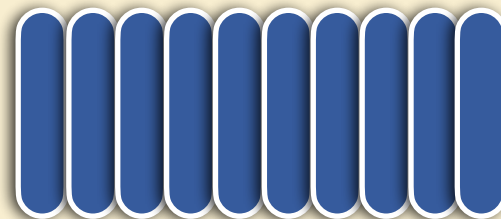
连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销，所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了。

对应的内存也要被分割成“物理块”或“页框”，这样就可以放下“页”
被分割成的固定大小的区域被称为“页”或“页面”

分页存储下的内存



地址变换机构



用户程序1

内存的分配



连续分配

离散分配

分页存储

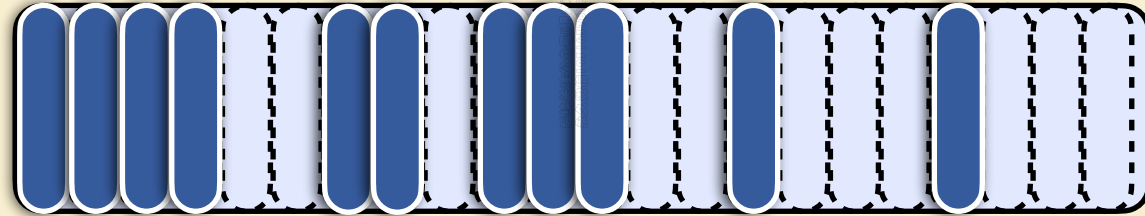
分段存储

段页式存储

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销，所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了。

对应的内存也要被分割成“物理块”或“页框”，这样就可以放下“页”
被分割成的固定大小的区域被称为“页”或“页面”

分页存储下的内存



用户程序1

地址变换机构

内存的分配



连续分配

离散分配

分页存储

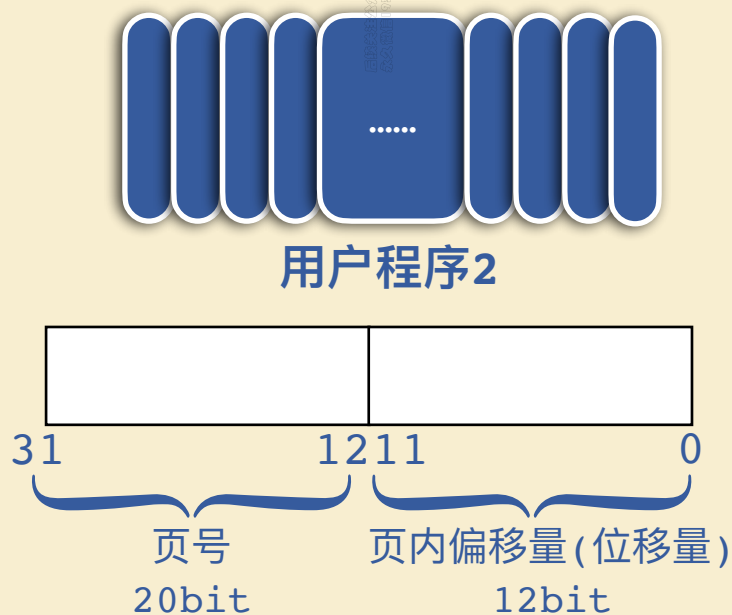
分段存储

段页式存储

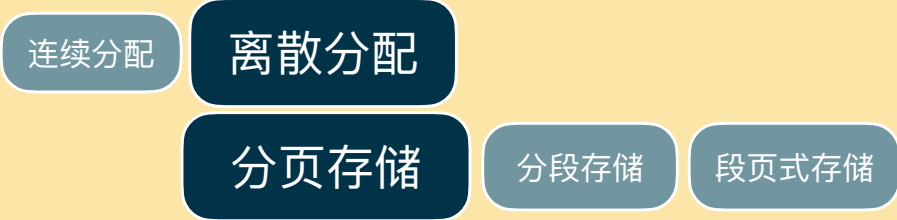
连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销，所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了。

分页地址中的地址结构（逻辑地址）

假设每页大小 $4KB = 2^{12}B$ ，共有 $1M = 2^{20}$ 页



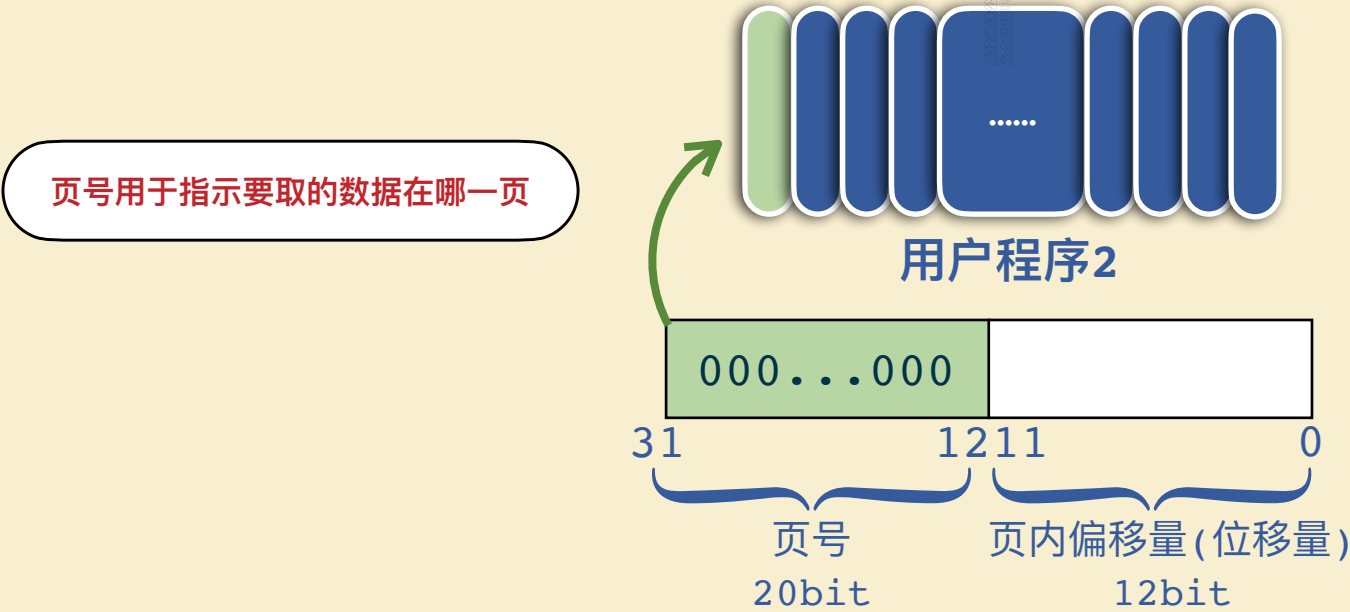
内存的分配



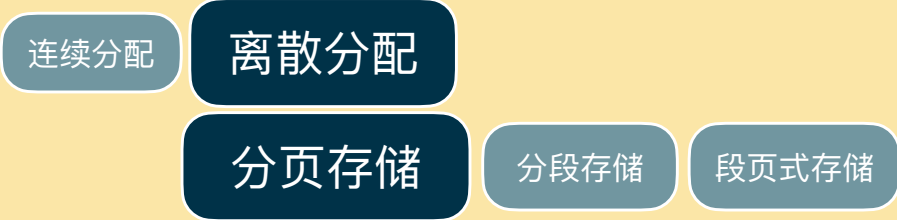
连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销，所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了。

分页地址中的地址结构（逻辑地址）

假设每页大小 $4KB = 2^{12}B$ ，共有 $1M = 2^{20}$ 页



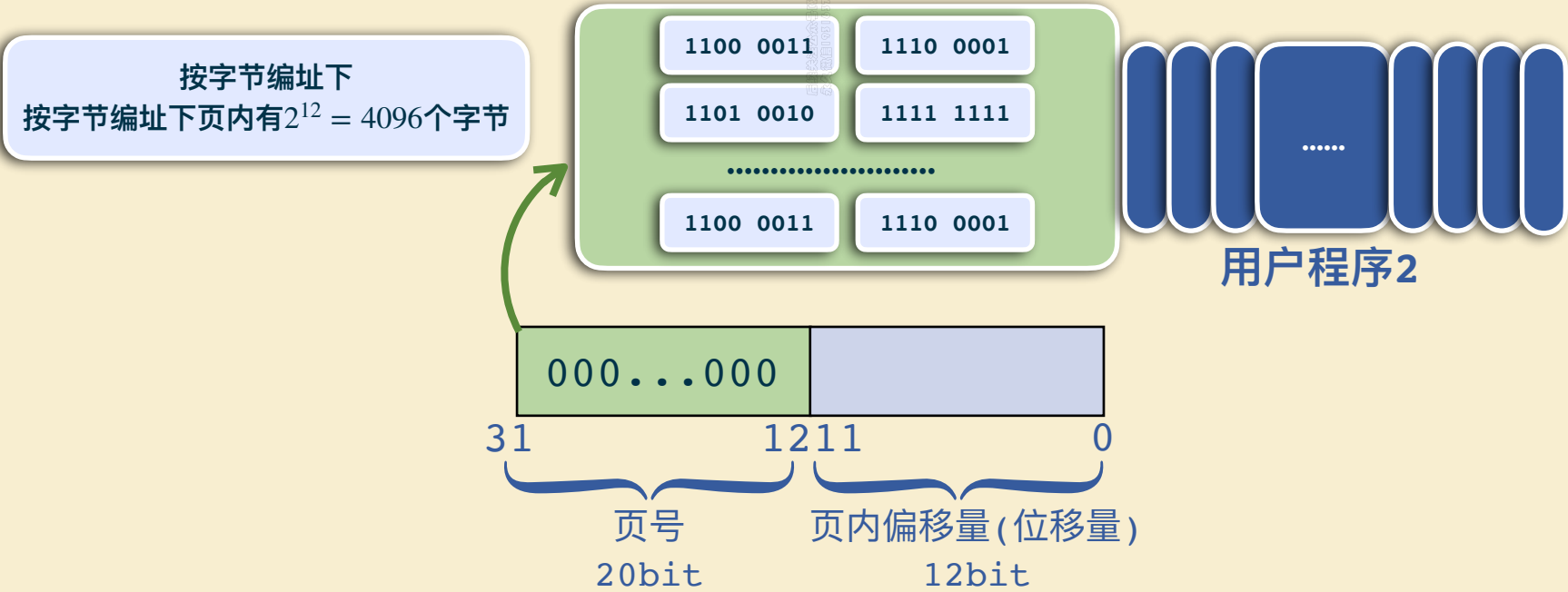
内存的分配



连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销，所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了。

分页地址中的地址结构（逻辑地址）

假设每页大小 $4KB = 2^{12}B$ ，共有 $1M = 2^{20}$ 页



内存的分配



连续分配

离散分配

分页存储

分段存储

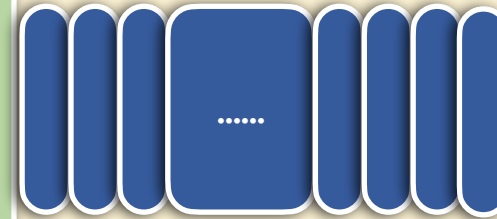
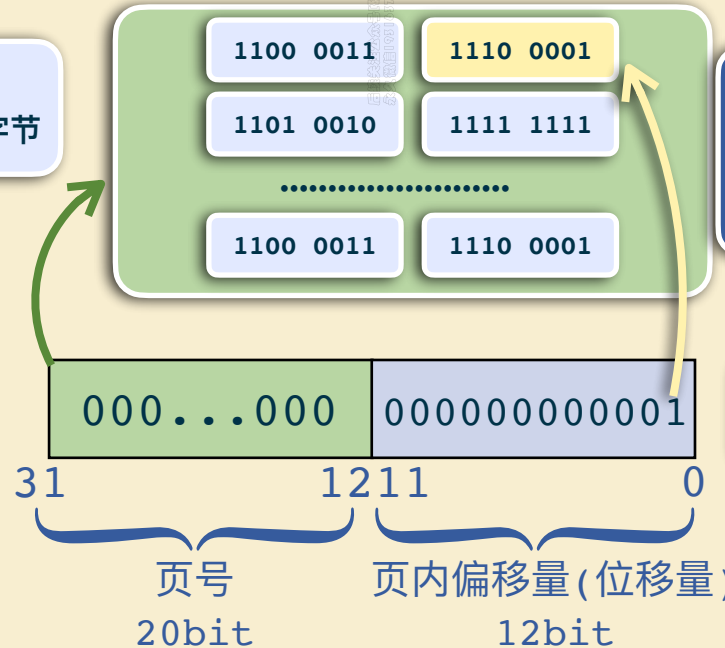
段页式存储

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销，所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了。

分页地址中的地址结构（逻辑地址）

假设每页大小 $4KB = 2^{12}B$ ，共有 $1M = 2^{20}$ 页

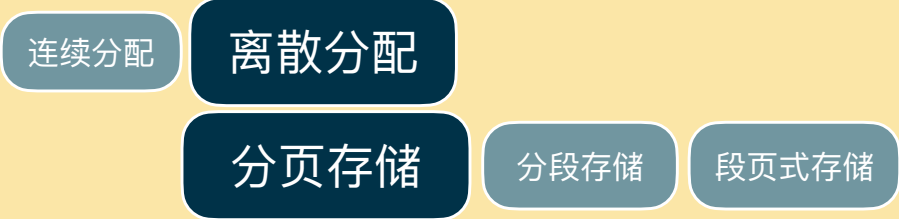
按字节编址下
按字节编址下页内有 $2^{12} = 4096$ 个字节



用户程序2

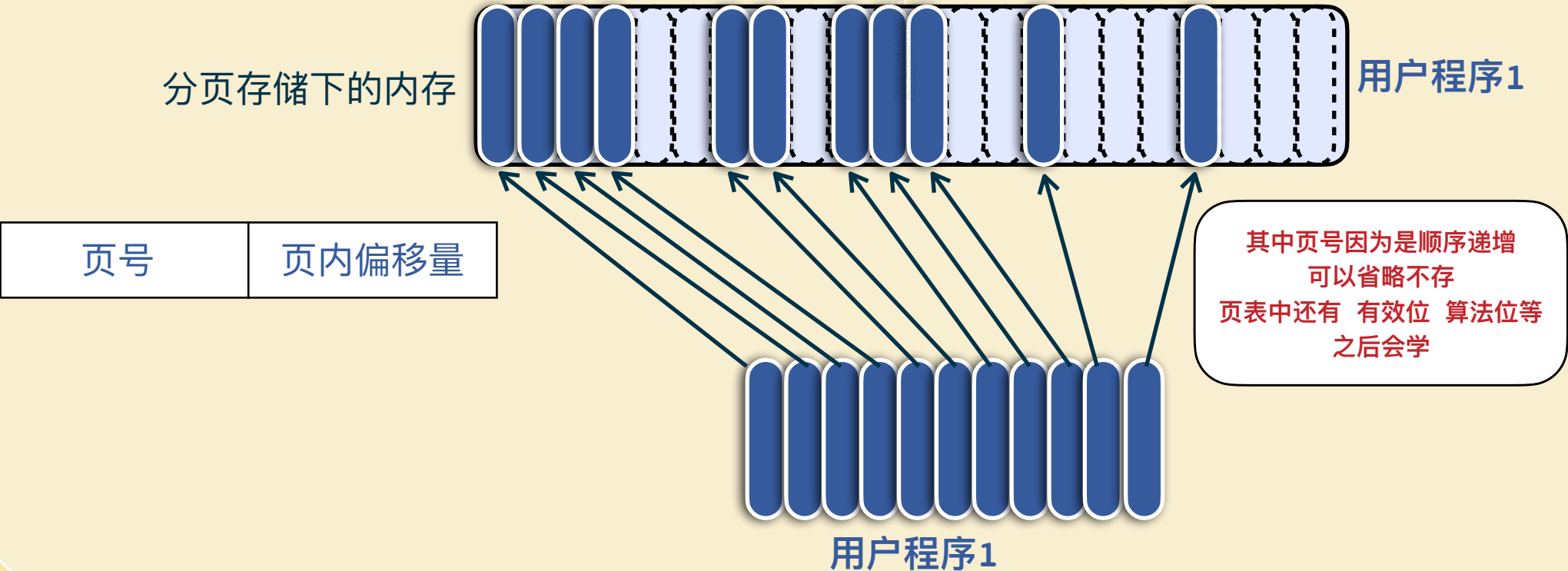
页内偏移量指出具体需要页内的哪一字节数据

内存的分配



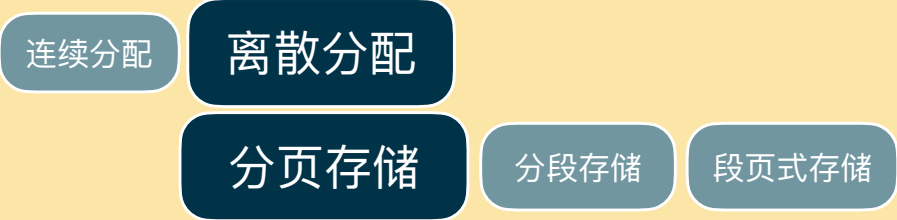
连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销，所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了。

每一页都可能被装进任意一个块，
所以需要一张表记录他们的映射关系



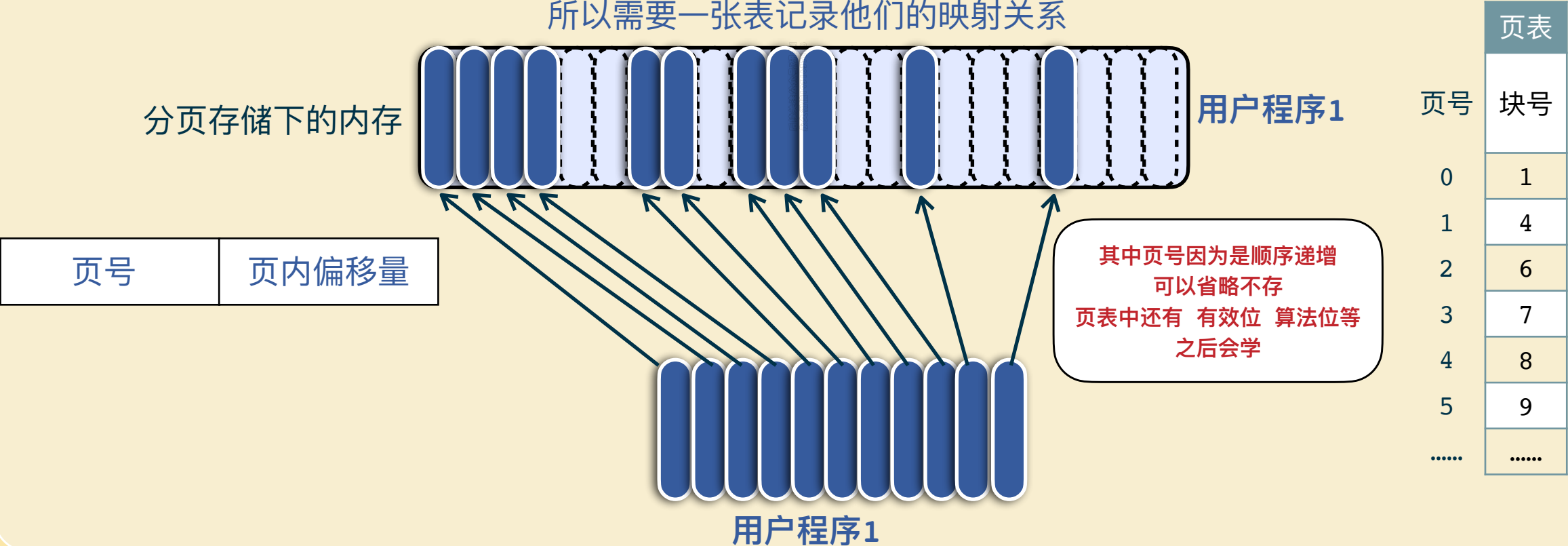
页表	
页号	块号
0	1
1	4
2	6
3	7
4	8
5	9
.....	

内存的分配



连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销，所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了。

每一页都可能被装进任意一个块，
所以需要一张表记录他们的映射关系



内存的分配



连续分配

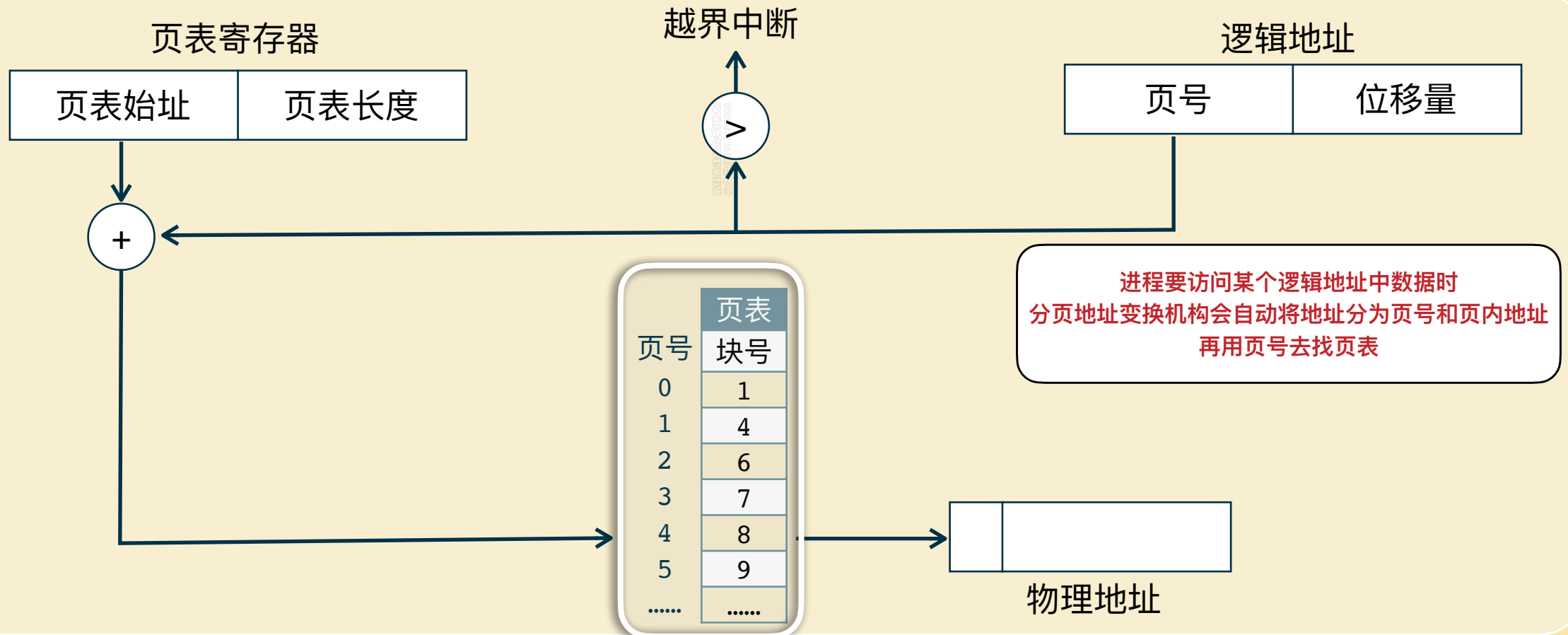
离散分配

分页存储

分段存储

段页式存储

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销，所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了。



内存的分配



连续分配

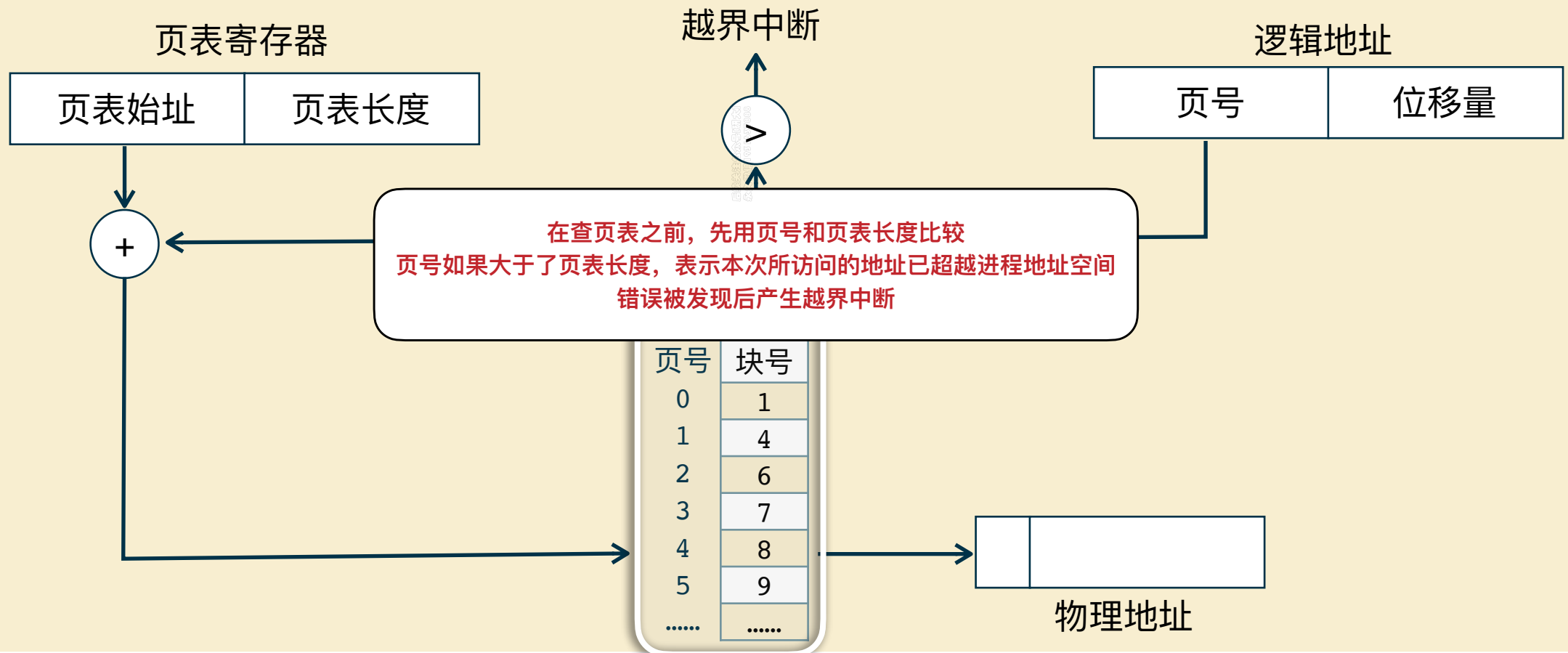
离散分配

分页存储

分段存储

段页式存储

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销，所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了。



内存的分配



连续分配

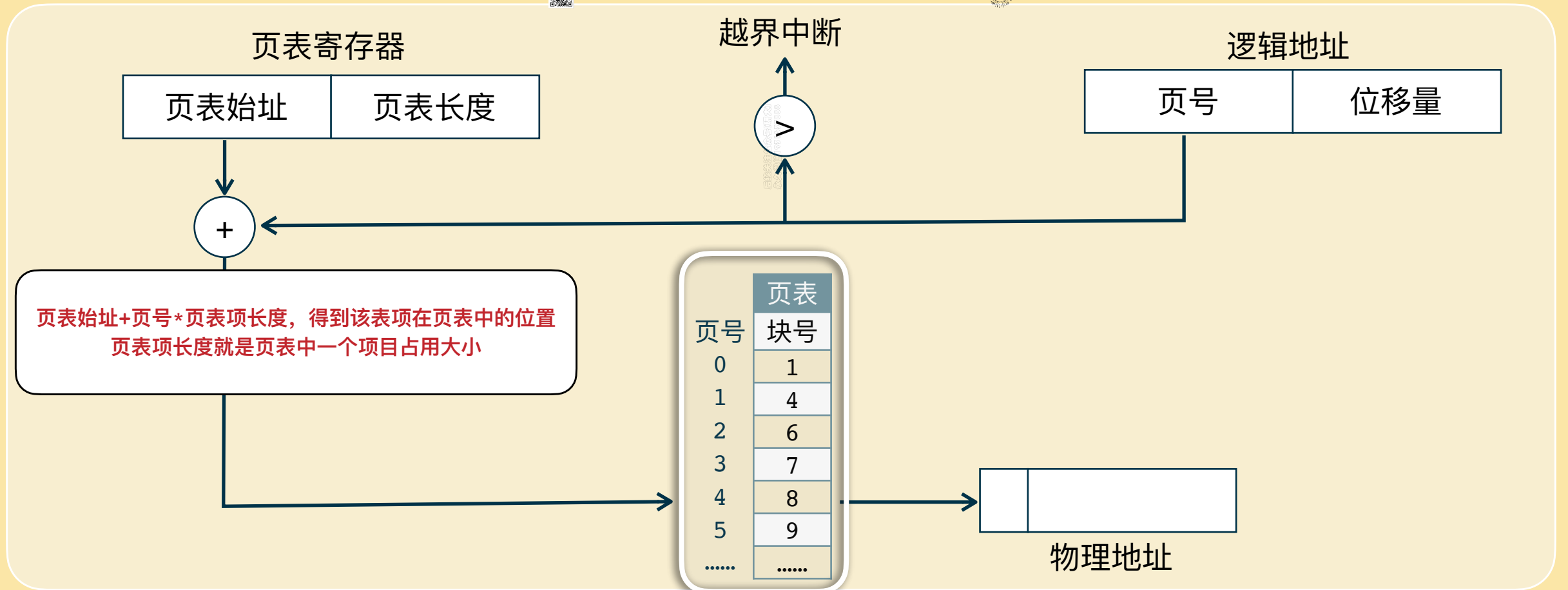
离散分配

分页存储

分段存储

段页式存储

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销，所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了。



内存的分配



连续分配

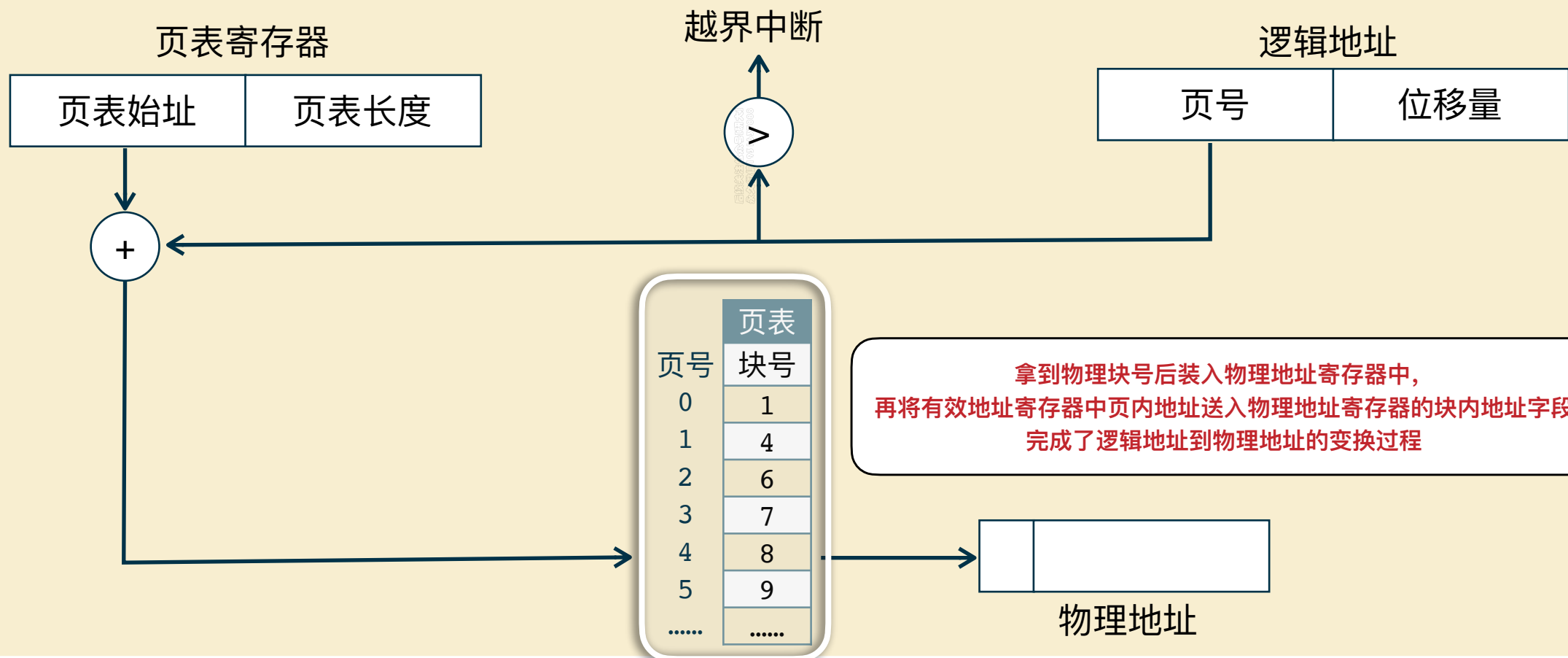
离散分配

分页存储

分段存储

段页式存储

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销，所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了。



内存的分配



连续分配

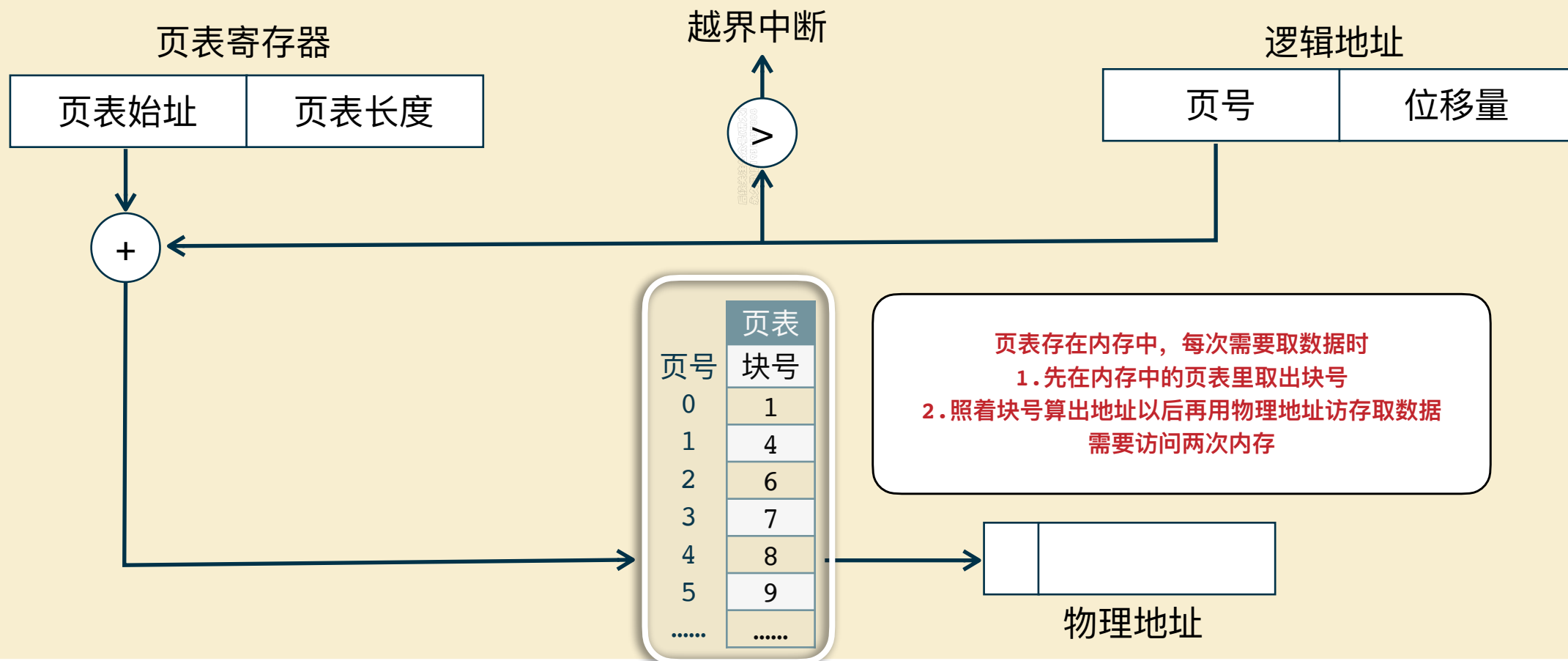
离散分配

分页存储

分段存储

段页式存储

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销，所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了。



内存的分配



连续分配

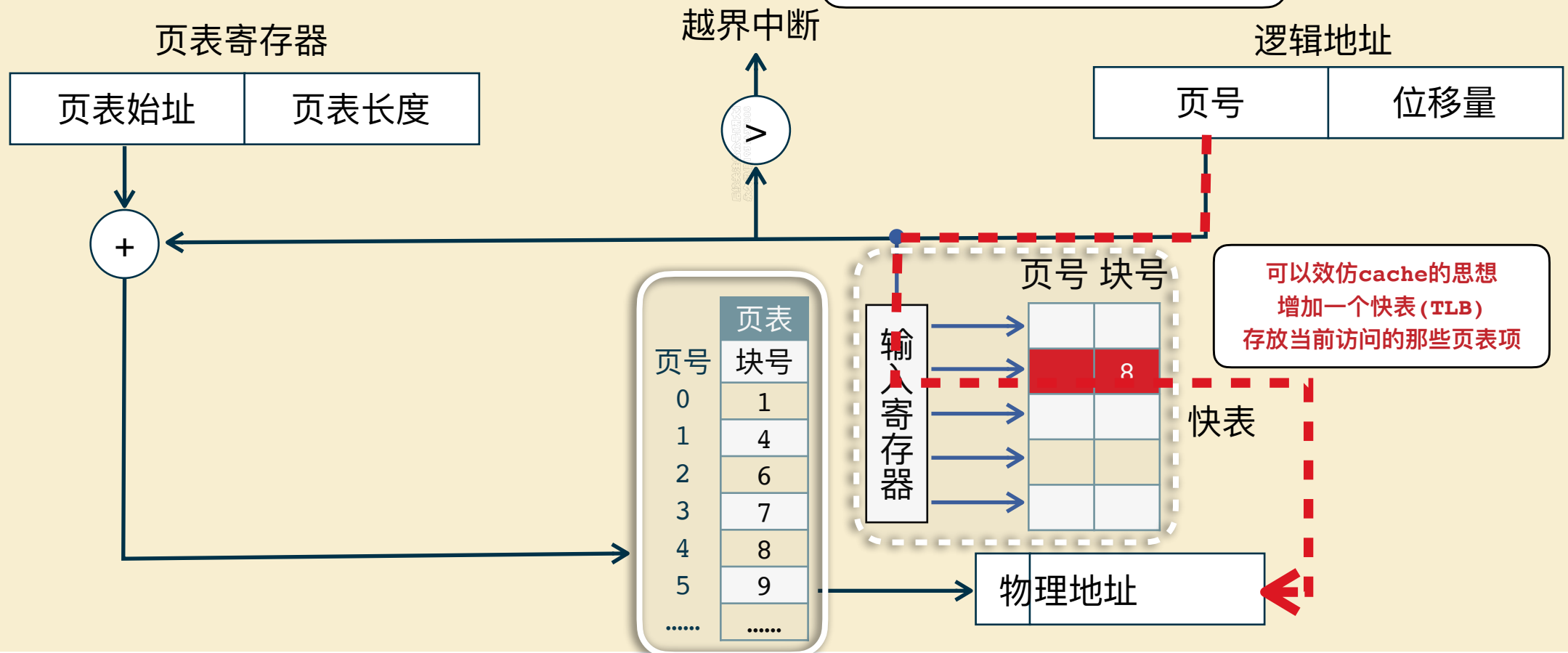
离散分配

分页存储

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片整理，所以不如直接把进程切碎了分散地装进许多不相邻的区域

CPU给出地址后，取出页号，与TLB中页号进行比较，若匹配成功可直接拿到块号拼成物理地址，随后拿着物理地址直接访问内存取数据，这样少了一次访问内存的时间

之付出很大开销，无须“紧凑”了



内存的分配



连续分配

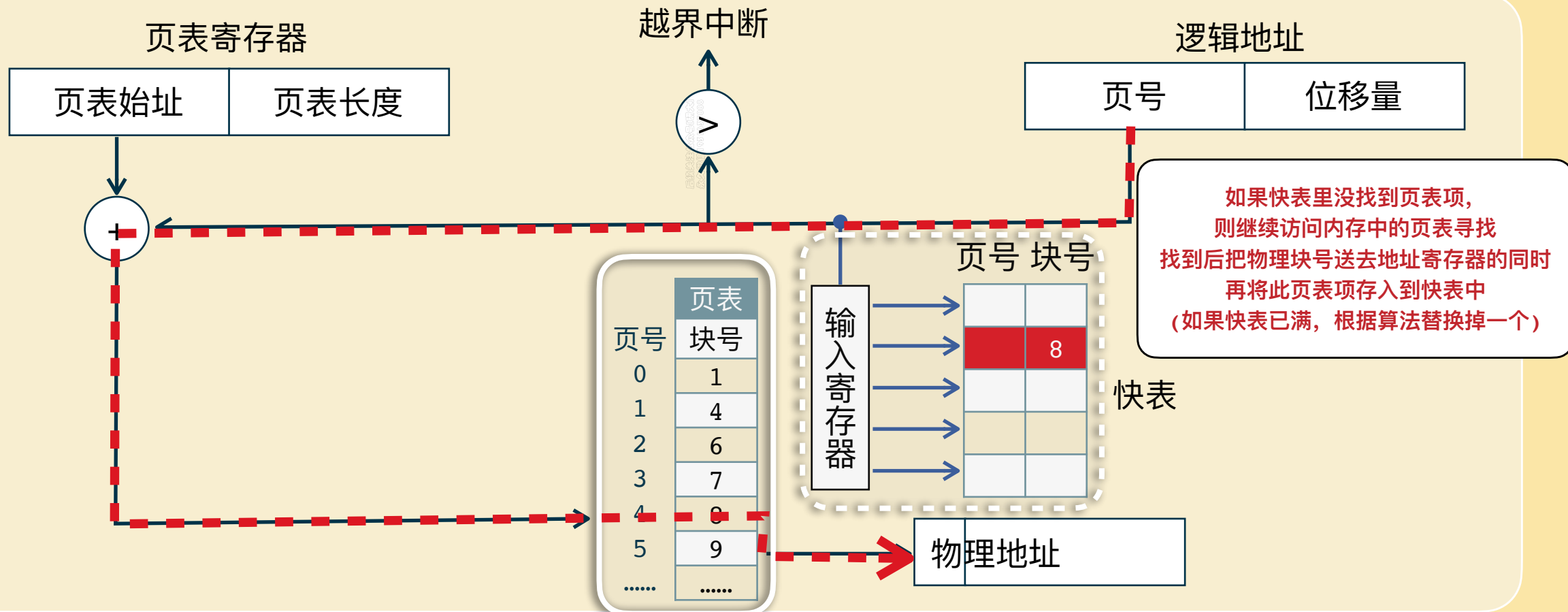
离散分配

分页存储

分段存储

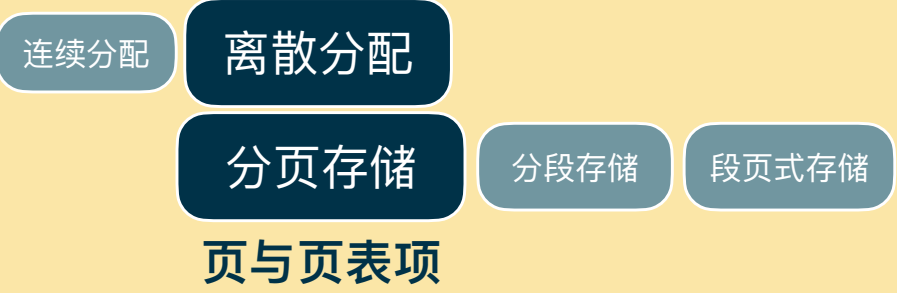
段页式存储

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销，所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了。





内存的分配



1. 页表是由很多个页表项组成的
右边**页表中每一行都是一个页表项**

2. 每个页表项中记录了一个页面被存在了哪一个物理块里

页表	
页号	块号
0	1
1	4
2	6
3	7
4	8
5	9
.....	

内存的分配



连续分配

离散分配

分页存储

分段存储

段页式存储

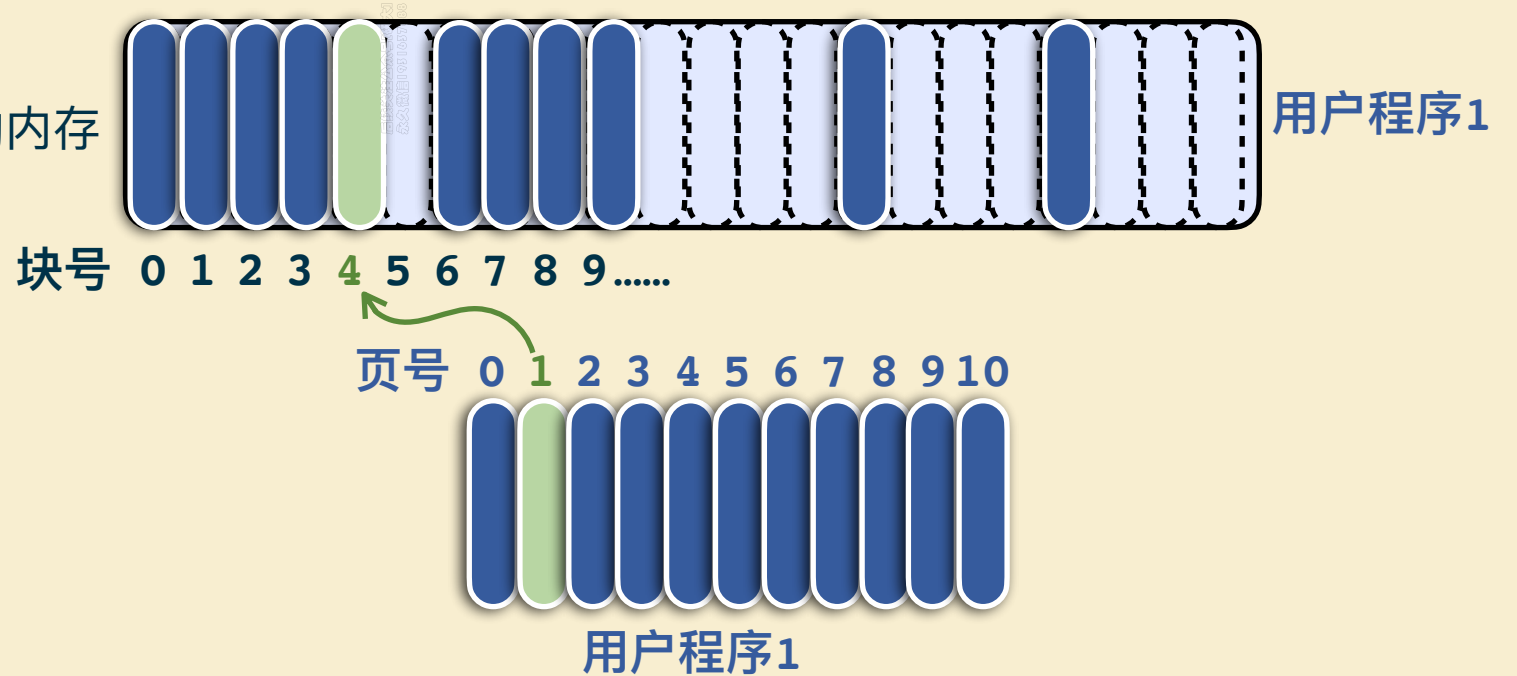
页与页表项

1. 页表是由很多个页表项组成的
右边**页表中每一行都是一个页表项**

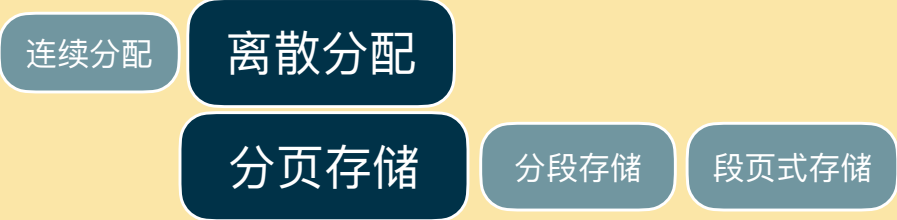
2. 每个页表项中记录了一个页面被存在了哪一个物理块里

页表	
页号	块号
0	1
1	4
2	6
3	7
4	8
5	9
.....	

分页存储下的内存



内存的分配



二级和多级页表

现代计算机都支持非常大的逻辑地址空间 ($2^{32} \sim 2^{64} \text{B}$)

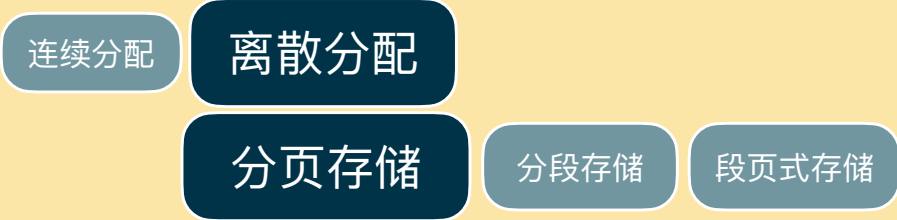
在这样的环境下页表就变得非常大，也会占用掉巨大内存空间

假设有32位逻辑地址，页面大小为4KB(2^{12}B)，则有20bit用来表示页号，即有 2^{20} 页

每个页表项占用一个字节，则每个进程仅页表就要占用1MB的连续的内存空间

页号	块号		块号		块号		块号	
0	1	9	21	18	114		1048567	114
1	4	10	23	19	115		1048568	115
2	6	11	25	20	121		1048569	121
3	7	12	26	21	133		1048570	133
4	8	13	28	22	159		1048571	159
5	9	14	30	23	140		1048572	140
6	15	15	33	24	155		1048573	155
7	2	16	40	25	177		1048674	177
8	3	17	55	26	180		1048575	180

内存的分配



二级和多级页表

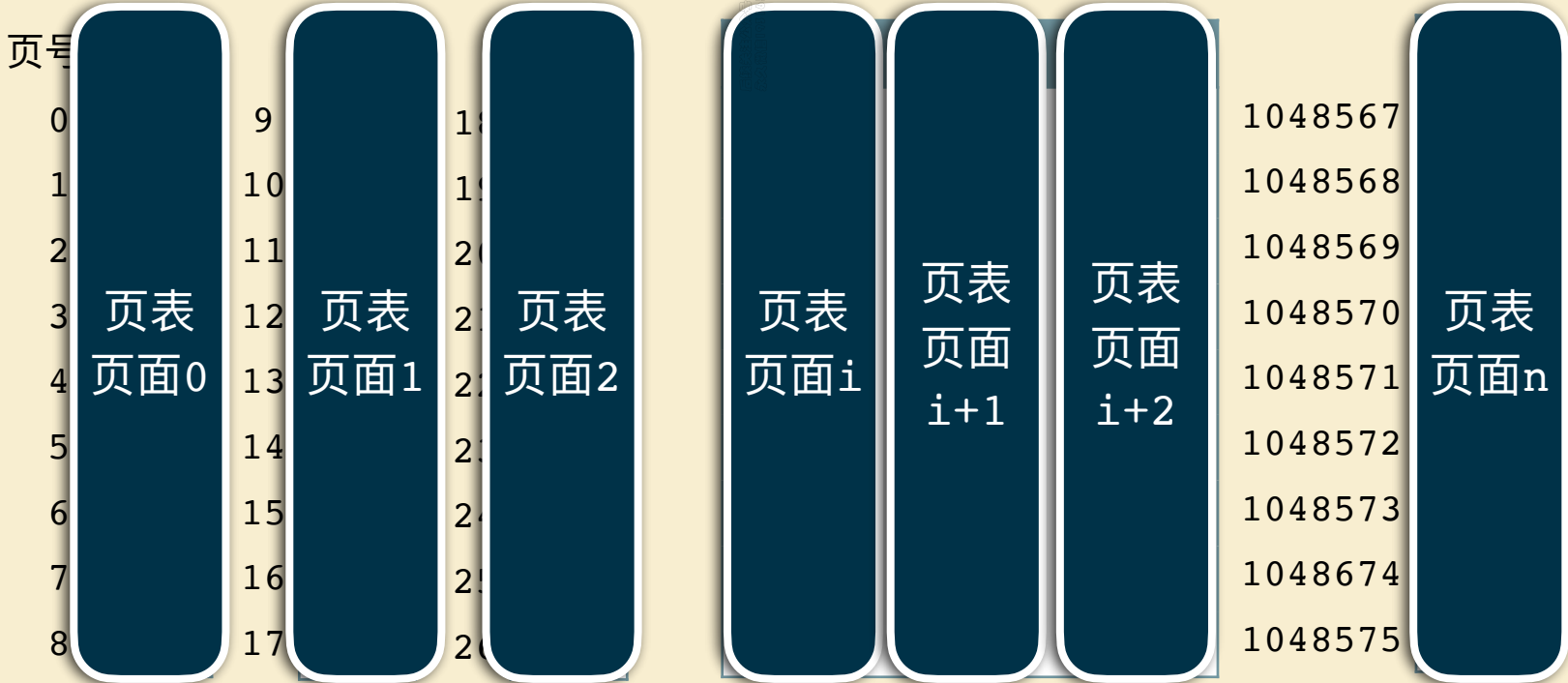
现代计算机都支持虚拟地址空间 ($2^{32} \sim 2^{64} \text{B}$)

在这样的环境下页表会占用巨大内存空间

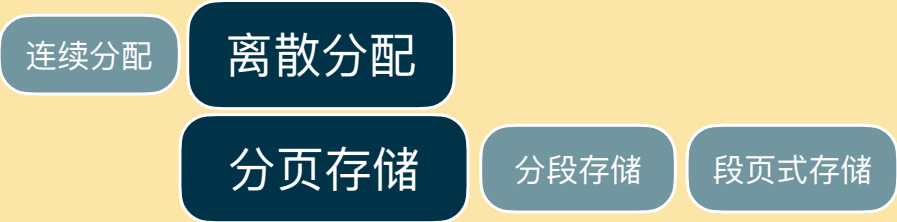
假设有32位逻辑地址，页面大小4KB，用20bit用来表示页号，即有 2^{20} 页

每个页表项占用一个字节，则每个进程仅页表就要占用1MB的连续的内存空间

可以把页表再次分页
减少占据的连续内存空间



内存的分配



二级和多级页表

现代计算机都支持虚拟地址空间 ($2^{32} \sim 2^{64} \text{B}$)

在这样的环境下页表会占用巨大内存空间

假设有32位逻辑地址，页面大小4KB，20bit用来表示页号，即有 2^{20} 页

每个页表项占用一个字节，则每个进程仅页表就要占用1MB的连续的内存空间

可以把页表再次分页
减少占据的连续内存空间

页目录表
(顶级页表)

页表编号	块号
0	1011
1	1078
2	...
3	...
...	...
n	1742

二级页表



存着具体数据的页面



内存的分配



连续分配

离散分配

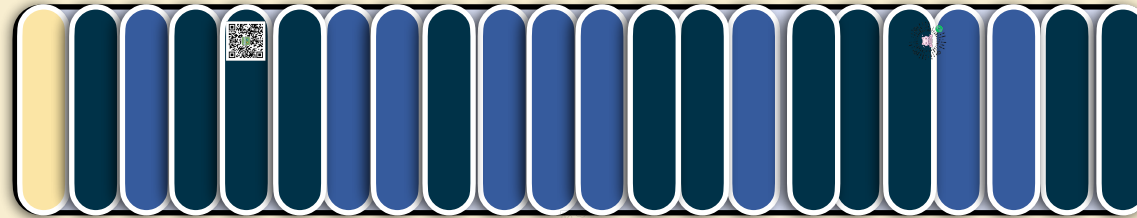
分页存储

分段存储

段页式存储

二级和多级页表

多级分页存储下的内存



页目录表常驻内存

二级页表离散存放

页面离散存放

页目录表 (顶级页表)

页表编号	块号
0	1011
1	1078
2	...
3	...
...	...
n	1742

二级页表

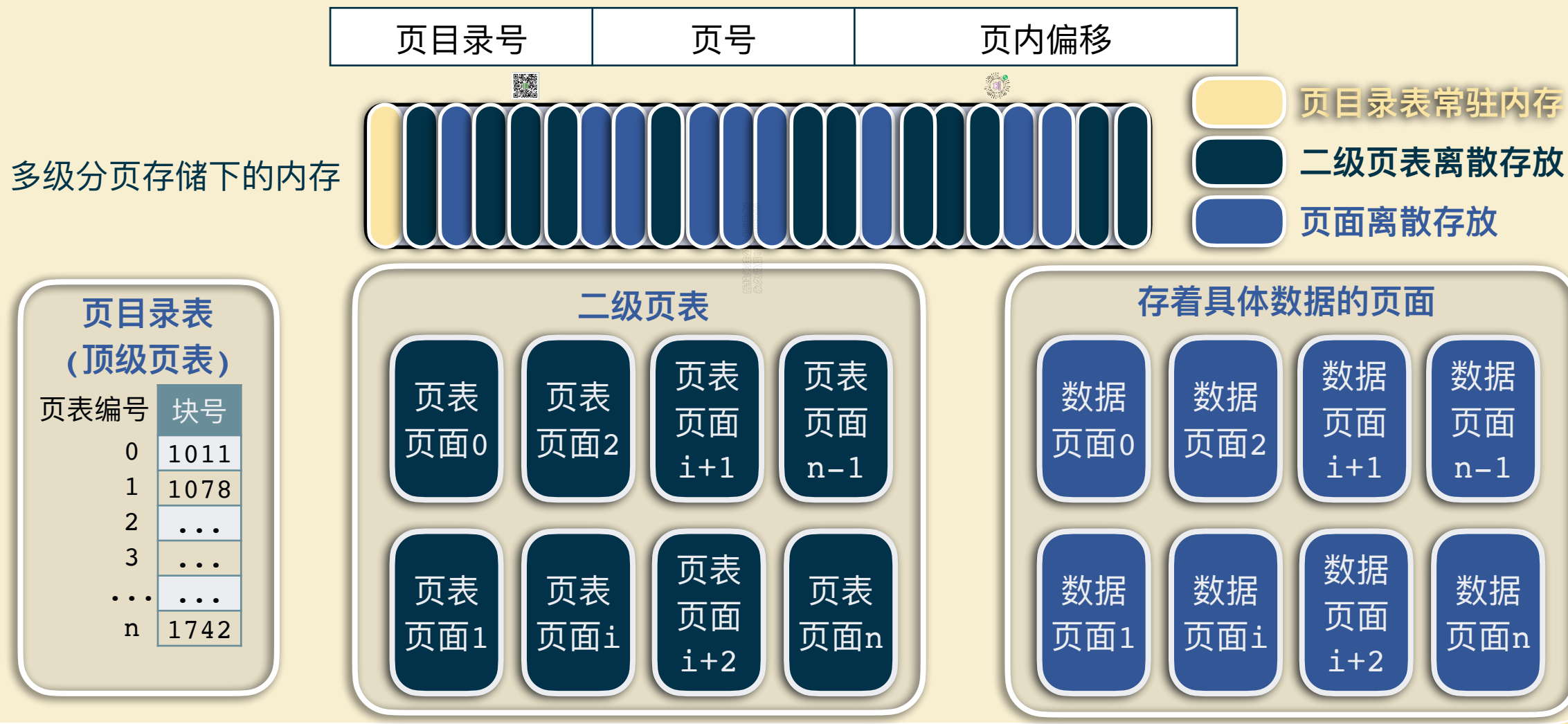


存着具体数据的页面





二级和多级页表





连续分配

离散分配

分页存储

分段存储

段页式存储

由页目录号查顶级页表
得到存放二级页表的物理块号(页框号)

二级和多级页表

页目录号

页号

页内偏移

多级分页存储下的内存

页目录表常驻内存

二级页表离散存放

页面离散存放

页目录表
(顶级页表)

页表编号	块号
0	1011
1	1078
2	...
3	...
...	...
n	1742

二级页表

页表
页面0页表
页面2页表
页面
 $i+1$ 页表
页面
 $n-1$ 页表
页面1页表
页面*i*页表
页面
 $i+2$ 页表
页面
 n

存着具体数据的页面

数据
页面0数据
页面2数据
页面
 $i+1$ 数据
页面
 $n-1$ 数据
页面1数据
页面*i*数据
页面
 $i+2$ 数据
页面
 n

内存的分配

连续分配

离散分配

分页存储

分段存储

段页式存储

由页目录号查顶级页表
得到存放二级页表的物理块号 (页框号)

一级和二级页表

页目录号	页号	页内偏移
------	----	------

- 页目录表常驻内存
- 二级页表离散存放
- 页面离散存放

多级分页存储下的内存

页目录表
(顶级页表)

页表编号	块号
0	1011
1	1078
2	...
3	...
...	...
n	1742

二级页表

页表
页面0

页表
页面2

页表
页面i+1

页表
页面n-1

页表
页面1

页表
页面i

页表
页面i+2

页表
页面n

存着具体数据的页面

数据
页面0

数据
页面2

数据
页面i+1

数据
页面n-1

数据
页面1

数据
页面i

数据
页面i+2

数据
页面n

连续分配

离散分配

分页存储

分段存储

段页式存储

由页目录号查顶级页表
得到存放二级页表的物理块号 (页框号)

二级和多级页表

页目录号	页号	页内偏移
------	----	------



- 页目录表常驻内存
- 二级页表离散存放
- 页面离散存放

多级分页存储下的内存

页目录表
(顶级页表)

页表编号	块号
0	1011
1	1078
2	...
3	...
...	...
n	1742

二级页表



存着具体数据的页面



- 连续分配
- 离散分配
- 分页存储
- 分段存储
- 段页式存储

由页目录号查顶级页表
得到存放二级页表的物理块号 (页框号)

二级和多级页表

页目录号	页号
------	----

二级页表中仍然有非常多页表项
(具体数量是一页的大小÷一个页表项大小)
通过页号找到要查的那个页面的物理块号 (页框号)

多级分页存储下的内存

页目录表 (顶级页表)

页表编号	块号
0	1011
1	1078
2	...
3	...
...	...
n	1742

二级页表

页表页面0

页表页面2

页表页面i+1

页表页面n-1

页表页面1

页表页面i

页表页面i+2

页表页面n

页号	块号
0	114
1	115
2	121
3	133
4	159
5	140
6	155
7	177
8	180

二级页表离散存放
页面离散存放

具体数据的页面

数据页面0

数据页面2

数据页面i+1

数据页面n-1

数据页面1

数据页面i

数据页面i+2

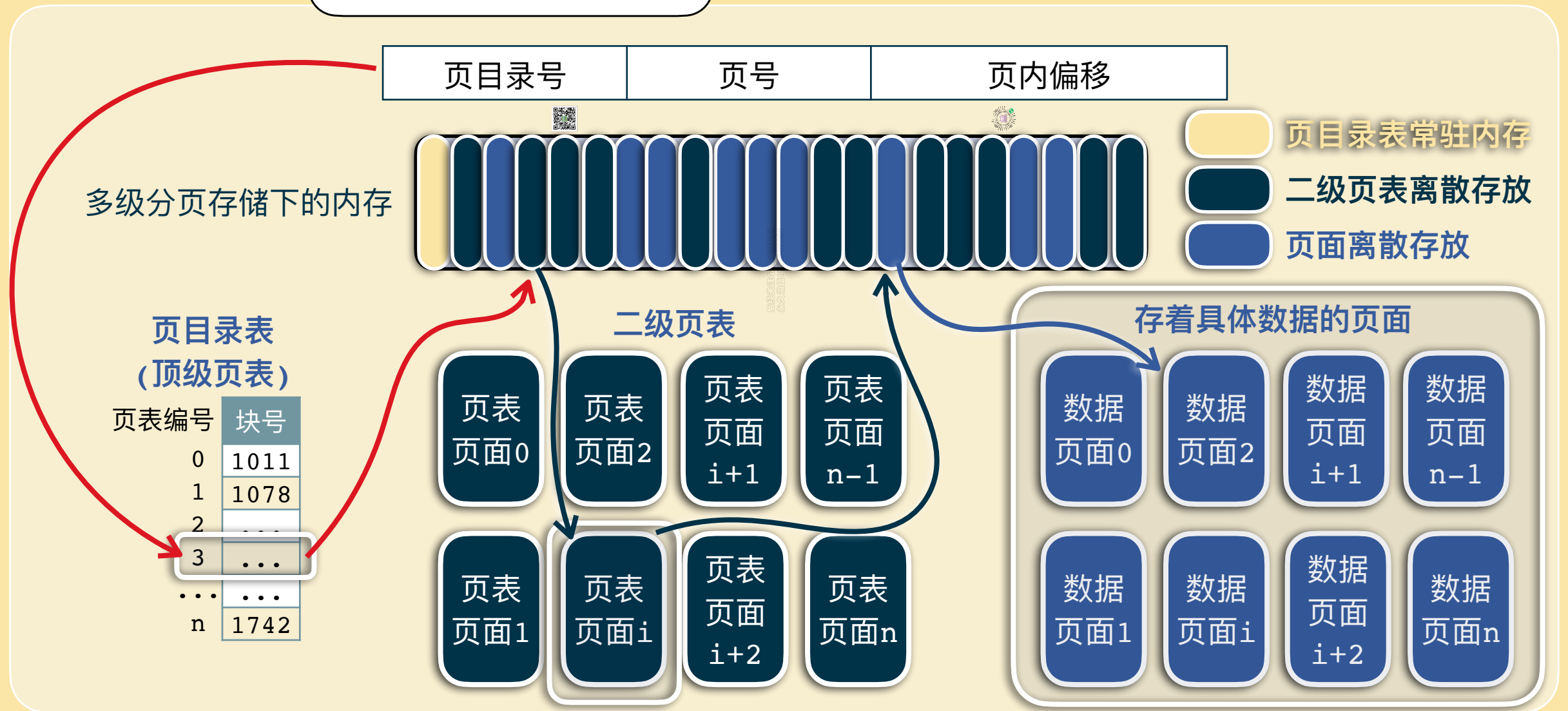
数据页面n

内存的分配

- 连续分配
- 离散分配
- 分页存储
- 分段存储
- 段页式存储

由页目录号查顶级页表
得到存放二级页表的物理块号 (页框号)

二级和三级页表



内存的分配

连续分配

离散分配

分页存储

分段存储

段页式存储

由页目录号查顶级页表
得到存放二级页表的物理块号 (页框号)

二级和多级页表

页目录号	页号	页内偏移
------	----	------

- 页目录表常驻内存
- 二级页表离散存放
- 页面离散存放

多级分页存储下的内存

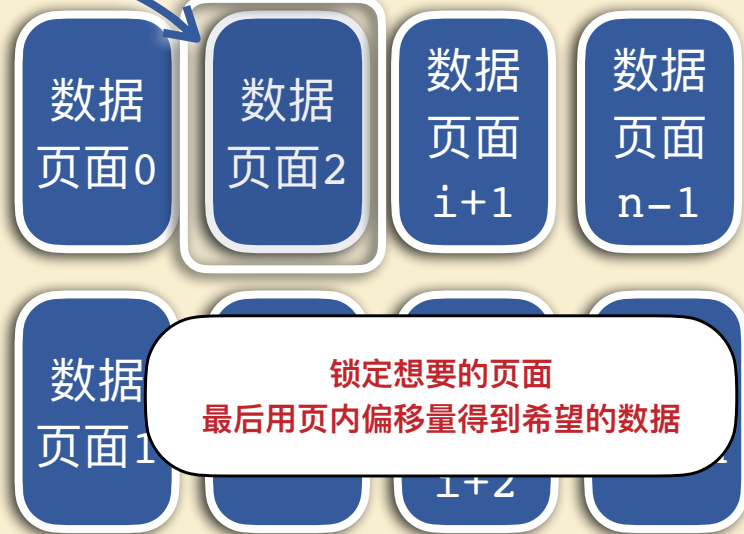
页目录表
(顶级页表)

页表编号	块号
0	1011
1	1078
2	...
3	...
...	...
n	1742

二级页表



存着具体数据的页面



锁定想要的页面
最后用页内偏移量得到希望的数据

内存的分配

连续分配

离散分配

分页存储

分段存储

段页式存储

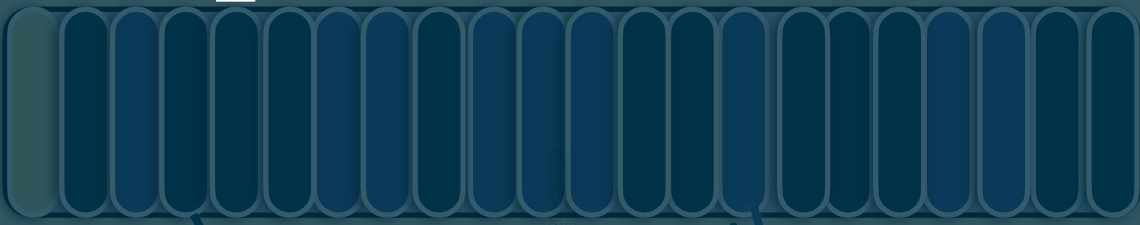
由页目录号查顶级页表
得到存放二级页表的物理块号 (页框号)

一级和二级页表

页目录号	页号	页内偏移
------	----	------

假设页内偏移量为01

多级分页存储下的内存

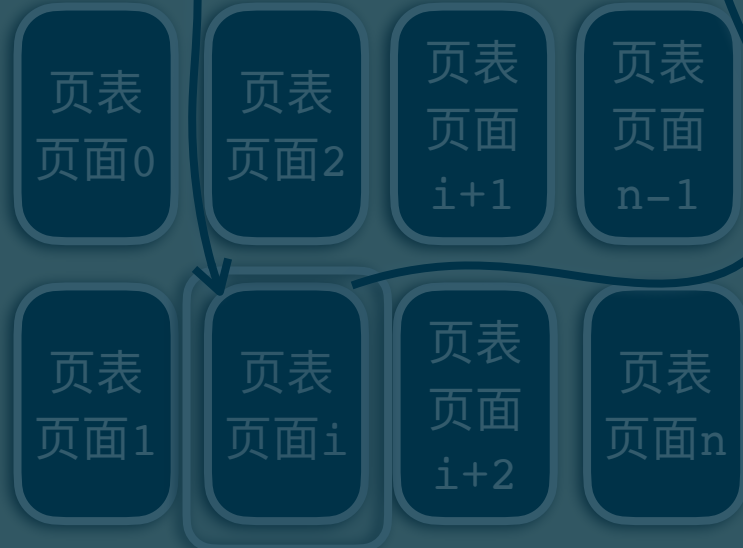


- 页目录表常驻内存
- 二级页表离散存放
- 页面离散存放

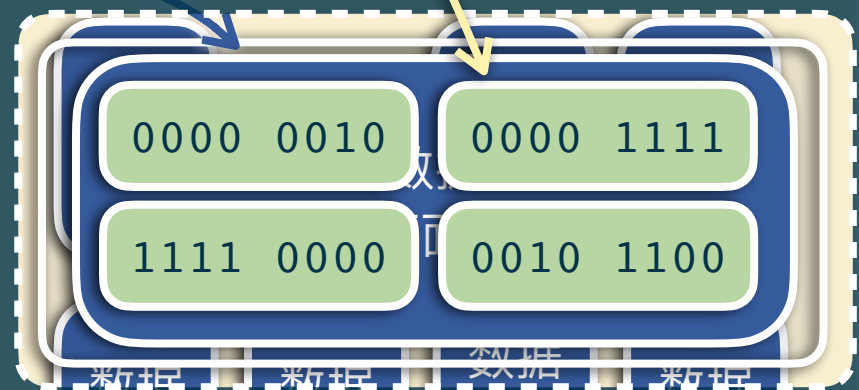
页目录表
(顶级页表)

页表编号	块号
0	1011
1	1078
2	...
3	...
...	...
n	1742

二级页表



存着具体数据的页面



锁定想要的页面
最后用页内偏移量得到希望的数据

内存的分配



连续分配

离散分配

分页存储

分段存储

段页式存储

由页目录号查顶级页表
得到存放二级页表的物理块号(页框号)

一级和二级页表

页目录号

页号

页内偏移

假设页内偏移量为01

多级分页存储下的内存

页目录表常驻内存

二级页表离散存放

页面离散存放

页目录表
(顶级页表)

页表编号	块号
0	1011
1	1078
2	...
3	...
...	...
n	1742

二级页表

页表
页面0

页表
页面2

页表
页面
 $i+1$

页表
页面
 $n-1$

页表
页面1

页表
页面
 i

页表
页面
 $i+2$

页表
页面
 n

存着具体数据的页面

0000 0010

0000 1111

1111 000

最终得到想要的1字节数据

锁定想要的页面
最后用页内偏移量得到希望的数据

内存的分配

连续分配

离散分配

BOK操作系统



分页存储

分段存储

段页式存储

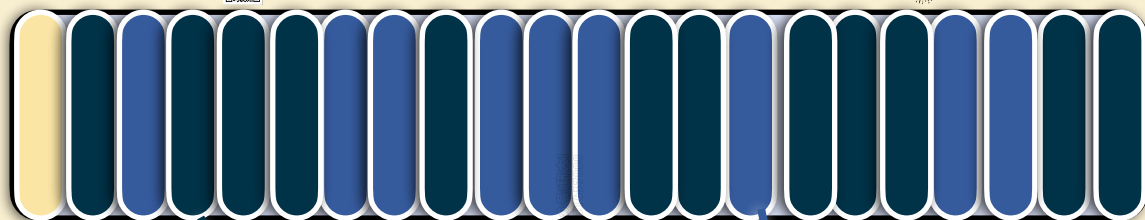
由页目录号查顶级页表
得到存放二级页表的物理块号 (页框号)

一级和二级页表

页目录号	页号	页内偏移
------	----	------

假设页内偏移量为01

多级分页存储下的内存



页目录表常驻内存

二级页表离散存放

页面离散存放

页目录表
(顶级页表)

页表编号	块号
0	1011
1	1078
2	...
3	...
...	...
n	1742

二级页表



存着具体数据的页面



最终得到想要的1字节数据

锁定想要的页面
最后用页内偏移量得到希望的数据



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

[2019]某计算机主存按字节编址，采用二级分页存储管理，地址结构如下

页目录号(10位)	页号(10位)	页内偏移(12位)
-----------	---------	-----------

虚拟地址2050 1225H对应的页目录号、页号分别是

- A.081H,101H
- B.081H,401H
- C.201H,101H
- D.201H,401H



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

[2019]某计算机主存按字节编址，采用二级分页存储管理，地址结构如下

页目录号(10位)	页号(10位)	页内偏移(12位)
-----------	---------	-----------

虚拟地址2050 1225H对应的页目录号、页号分别是

A.081H,101H

B.081H,401H

C.201H,101H

D.201H,401H



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

[2019]某计算机主存按字节编址，采用二级分页存储管理，地址结构如下

页目录号 (10位)	页号 (10位)	页内偏移 (12位)
------------	----------	------------

虚拟地址2050 1225H对应的页目录号、页号分别是



- A.081H,101H
- B.081H,401H
- C.201H,101H
- D.201H,401H



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

[2019]某计算机主存按字节编址，采用二级分页存储管理，地址结构如下

页目录号(10位)	页号(10位)	页内偏移(12位)
0010 0000 01	0100000001	001000100101

虚拟地址2050 1225H对应的页目录号、页号分别是

A.081H,101H

B.081H,401H

C.201H,101H

D.201H,401H



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

[2019]某计算机主存按字节编址，采用二级分页存储管理，地址结构如下

页目录号(10位)	页号(10位)	页内偏移(12位)
0010000001	0100000001	001000100101

虚拟地址2050 1225H对应的页目录号、页号分别是

- A.081H,101H
- B.081H,401H
- C.201H,101H
- D.201H,401H



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

[2019]某计算机主存按字节编址，采用二级分页存储管理，地址结构如下

页目录号(10位)	页号(10位)	页内偏移(12位)
00 1000 0001	01 0000 0001	0010 0010 0101
081H	101H	225H

虚拟地址2050 1225H对应的页目录号、页号分别是

- A.081H,101H
- B.081H,401H
- C.201H,101H
- D.201H,401H



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

[2010]某计算机采用二级页表的分页存储管理方式，按字节编址，页大小为 2^{10}B ，页表项大小为 2B ，逻辑地址结构为

页目录号	页号	页内偏移
------	----	------

逻辑地址空间大小为 2^{16} 页，则标识整个逻辑地址空间的页目录表中包含表项的个数至少是

- A. 64
- B. 128
- C. 256
- D. 512



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

[2010]某计算机采用二级页表的分页存储管理方式，按字节编址，页大小为 2^{10}B ，页表项大小为 2B ，逻辑地址结构为

页目录号	页号	页内偏移
------	----	------

逻辑地址空间大小为 2^{16} 页，则标识整个逻辑地址空间的页目录表中包含表项的个数至少是

A. 64

B. 128

C. 256

D. 512

内存的分配



之前介绍的分页机制通过硬件机制实现，对用户完全透明
分段管理方式则考虑了用户和程序员

分段系统将用户进程的逻辑地址空间划分为大小不等的段

每段都从0开始编址，并且会在内存中被分配一段连续的地址空间
(段内要求连续，段间不要求连续，进程的地址空间是二维的)

分段地址的地址结构



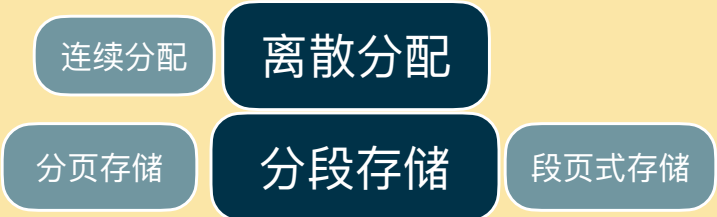
31 16 15 0

该地址允许一个作业最多有 2^{16} 个段
每个段最长 2^{16} B



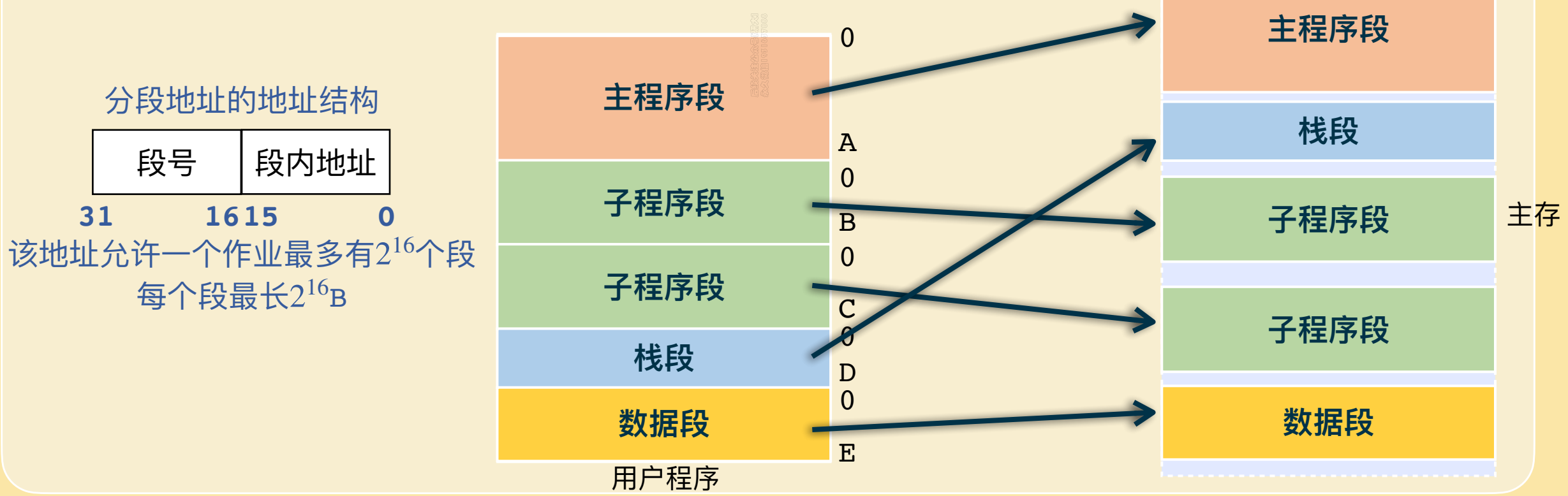
用户程序

内存的分配



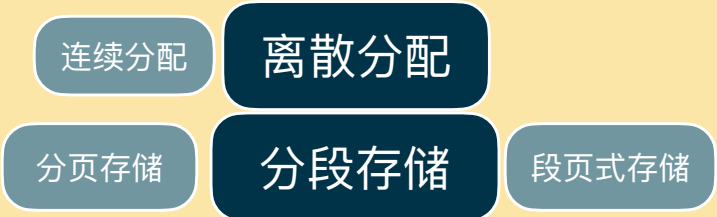
之前介绍的分页机制通过硬件机制实现，对用户完全透明
分段管理方式则考虑了用户和程序员

分段系统将用户进程的逻辑地址空间划分为大小不等的段
每段都从0开始编址，并且会在内存中被分配一段连续的地址空间
(段内要求连续，段间不要求连续，进程的地址空间是二维的)

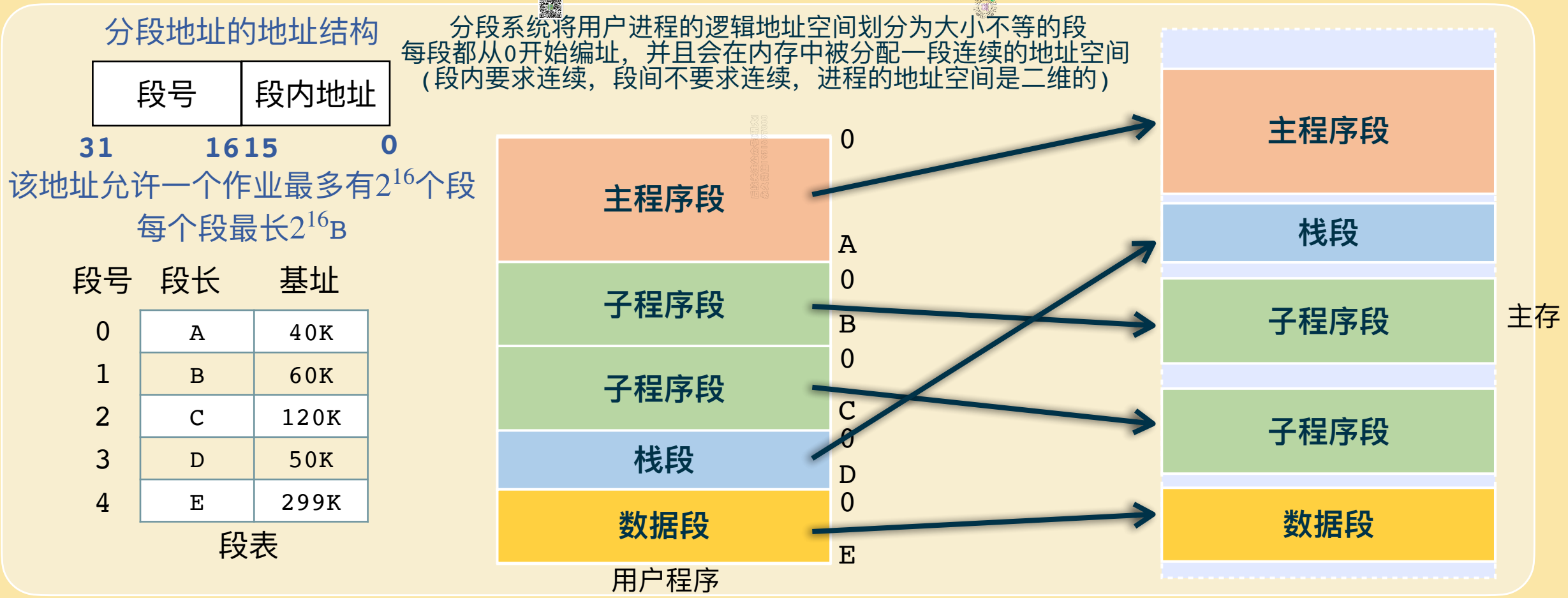


主存

内存的分配



之前介绍的分页机制通过硬件机制实现，对用户完全透明
分段管理方式则考虑了用户和程序员



内存的分配



连续分配

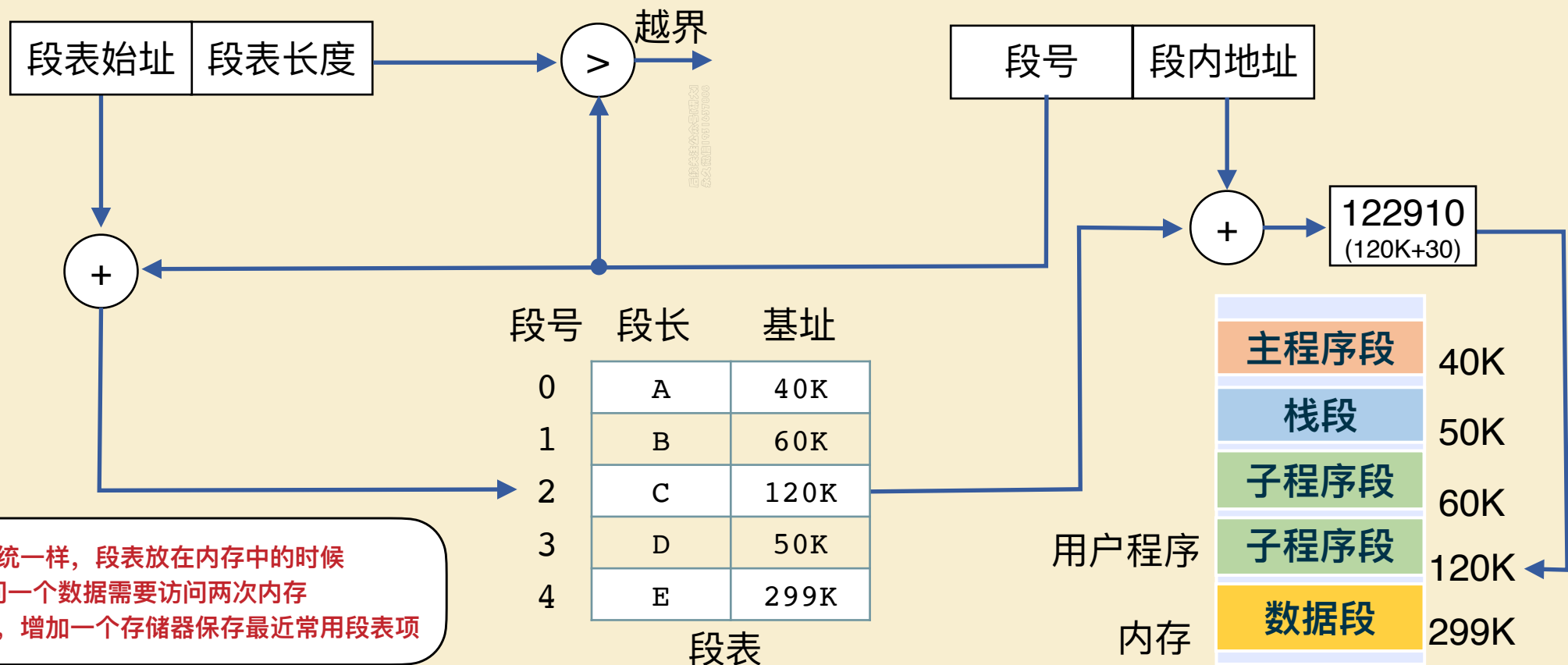
离散分配

分页存储

分段存储

段页式存储

之前介绍的分页机制通过硬件机制实现，对用户完全透明
分段管理方式则考虑了用户和程序员



内存的分配



连续分配

离散分配

分页存储

分段存储

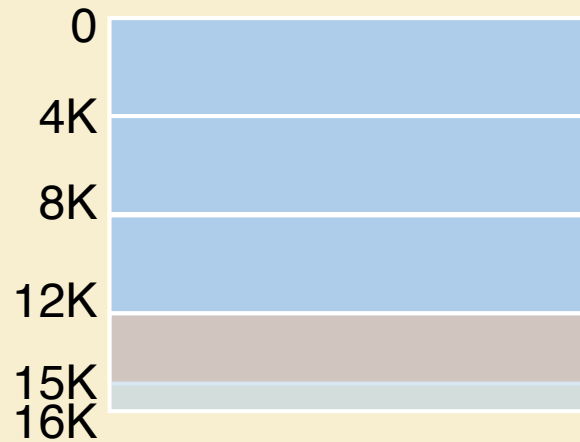
段页式存储



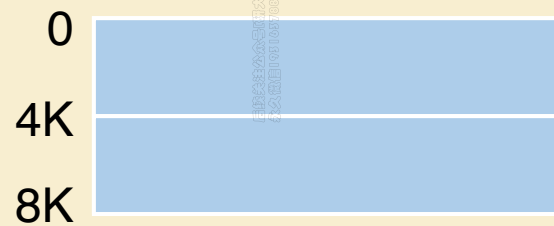
先分段再分页的缝合怪



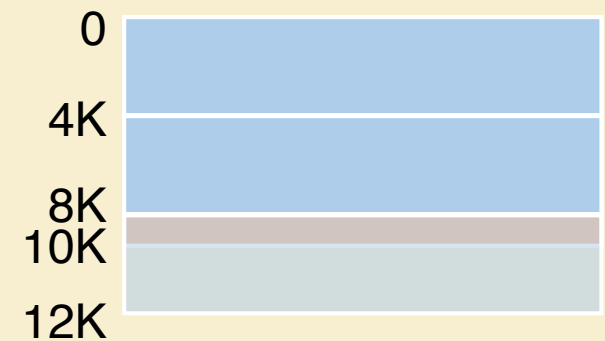
主程序段



子程序段



数据段



段号s

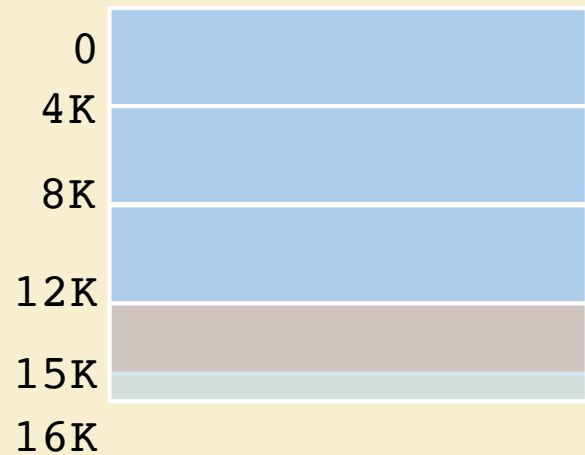
段内页号P

页内地址w

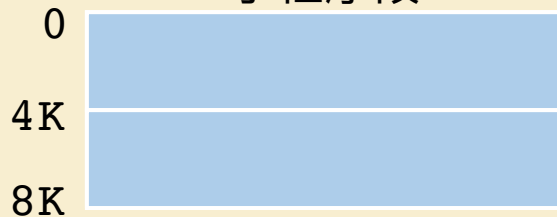
作业地址结构

先把用户程序分成段，再把每个段分成页，并为每个段赋予一个段名

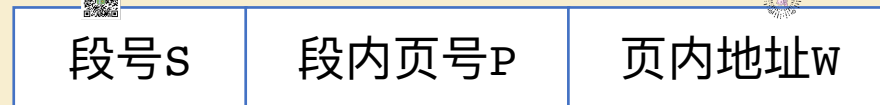
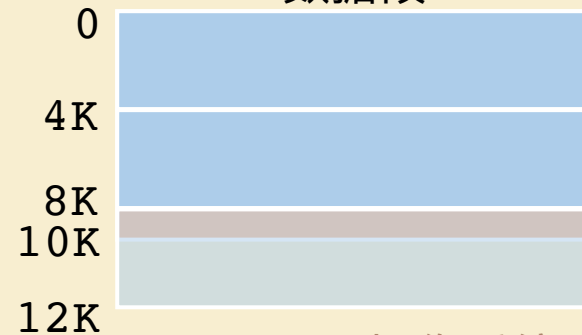
主程序段



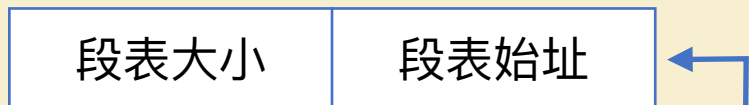
子程序段



数据段



作业地址结构

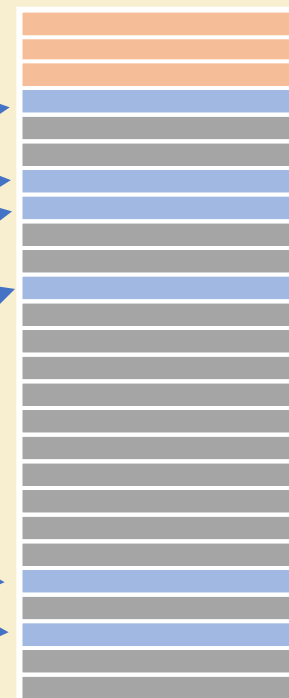


段号	状态	页表大小	页表始址
0	1		
1	1		
2	1		
3	0		
4	1		

页号	状态	存储块
0	1	
1	1	
2	1	
3	0	
4	1	

页号	状态	存储块

操作系统



内存空间

内存的分配



连续分配

离散分配

分页存储

分段存储

段页式存储

先分段再分页的缝合怪

